Faculty of Science

Software Engineering Teaching Unit

University of Kelaniya, Sri Lanka

SENG 31242 — System Design Project

SOFTWARE DESIGN SPECIFICATION

TeaRoutePay

Offline-First Tea Collection, Route Logistics, and Farmer Payment Management System

Field	Value
Document ID	TCPS-SDS-2026-v1.0
Date	June 2026
Status	FINAL
Version	1.0
Prepared by	Group 09 — SENG 31242 (2026)
Companion Document	TeaRoutePay SRS (TCPS-SRS-2026-v1.0)

Group 09

Registration Number	Name
SE/2022/009	Prabhath Harsha
SE/2022/024	Thisari Uresha
SE/2022/027	Shashini Wijesingha
SE/2022/032	Yehan Kanishka
SE/2022/044	Nihadh Nilabdeen

GitHub Organisation: https://github.com/seng31242-tea-route-pay-2026

TABLE OF CONTENTS

1. INTRODUCTION 5
1.1 Purpose 5
1.2 Scope 5
1.3 References 6
1.4 Overview of the Document 7
2. SYSTEM OVERVIEW 9
2.1 System Description and the Hybrid Local-First Architecture 9
2.2 Relationship to the SRS 10
3. ARCHITECTURAL DESIGN 12
3.1 Selected Architecture and Justification 12
3.2 High-Level Microservices Architecture 13
3.3 Service Responsibilities 14
3.4 Component Diagram 15
3.5 Deployment Diagram 16
3.6 Technology Stack 17
4. DETAILED DESIGN 19
4.1 Domain Class Diagram 19
4.2 Sequence Diagrams (Use-Case Realisations) 20
SD-01 — Log in to Mobile App (UC-01) 21
SD-02 — View Assigned Route (UC-02) 21
SD-03 — Scan Farmer QR Code (UC-03) 22
SD-04 — Record Leaf Weight (UC-04) 23
SD-05 — Item Distribution and Record Keeping (UC-05) 23
SD-06 — Issue Advance Payment and Record Keeping (UC-06) 24
SD-07 — Synchronise Collection Data (UC-07) 25
SD-08 — Log in to Desktop App (UC-08) 26
SD-09 — Register Farmer (UC-09) 27
SD-10 — Manage Farmer Profiles (UC-10) 28
SD-11 — Manage Deductions (UC-11) 28
SD-12 — Set Monthly Tea Rate (UC-12) 29
SD-13 — Calculate Monthly Payment (UC-13) 30
SD-14 — View Payment Summary (UC-14) 31
SD-15 — Approve Payment Run (UC-15) 32
SD-16 — Process Bank Transfer (UC-16) 32

SD-17 — Send SMS Notification (UC-17) *33*
SD-18 — Generate Reports (UC-18) *34*
SD-19 — Manage User Accounts (UC-19) *34*
SD-20 — Manage Collection Routes (UC-20) *35*
4.3 State Machine Diagrams 36
4.3.1 Farmer State Machine *36*
4.3.2 Payment Run State Machine *37*
4.3.3 Collection Record State Machine *38*
4.4 Database Design 38
4.4.1 Central (Cloud / Office) Entity-Relationship Model *38*
4.4.2 Mobile Local (SQLite) Schema *39*
4.4.3 Data Dictionary *40*
4.4.4 Architectural Data-Integrity Rules *41*
4.4.5 Requirements Traceability Matrix (Functional) *42*
5. USER INTERFACE DESIGN 46
5.1 UX Decisions for Low-Technical-Proficiency Users 46
5.1.1 Mobile Application Wireframes *47*
..... 48
..... 49
5.1.2 Office Desktop Application Wireframes *50*
..... 50
..... 51
..... 52
..... 53
5.2 Navigation Flow 54
5.3 UI Resources and Prototyping 55
6. INTERFACE DESIGN 56
6.1 External Interfaces 56
6.1.1 Bank Transfer API *56*
6.1.2 SMS Gateway API *57*
6.2 Internal Interfaces 57
6.2.1 Synchronisation Payload *58*
7. SECURITY DESIGN 58
7.1 Authentication and Authorisation 58
7.2 Data Protection 59
7.3 Auditability 60
7.4 OWASP Top 10 Mitigations 60
8. NON-FUNCTIONAL DESIGN DECISIONS 62
APPENDIX A — ARCHITECTURAL DECISION RECORDS 64

ADR-001: Adopt a Microservices Architecture 64
ADR-002: Use PostgreSQL as the Central Database 64
ADR-003: Use RabbitMQ for Asynchronous Event-Driven Communication 65
ADR-004: Offline-First Mobile Architecture with Encrypted SQLite 65

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Design Specification (SDS) is to provide a complete, unambiguous, and technically rigorous description of the design of the TeaRoutePay system, derived directly from the approved Software Requirements Specification (SRS, document TCPS-SRS-2026-v1.0). Where the SRS establishes what the system is required to do, the present document establishes how those requirements are to be realised in software.

This SDS is intended to serve as the authoritative engineering reference that bridges the analysis phase and the implementation phase of the project. It is written to be sufficient for the following purposes:

- to allow the development team to implement each subsystem without re-deriving design intent from the requirements;
- to allow reviewers, supervisors, and panel members to verify that every functional requirement (FR-01 to FR-53) and every non-functional requirement (NFR-01 to NFR-29) of the SRS is satisfied by a concrete, traceable design decision;
- to allow future maintainers, including the SENG 34213 development team, to understand the rationale behind each architectural and detailed-design choice; and
- to establish the verification baseline against which the constructed system will be tested.

A central design principle adopted throughout this document is explicit traceability. Each significant design decision is mapped back to the specific SRS requirement or requirements that motivate it. This traceability is expressed both inline — through Requirements addressed annotations — and consolidated in the requirements-traceability matrices presented in Sections 4 and 8.

1.2 Scope

TeaRoutePay is a distributed, offline-first software platform that digitises the complete operational workflow of a tea collection business, extending from field-level leaf collection

through to monthly farmer payment disbursement and confirmation. The design described in this document encompasses the following system boundary:

- **Collector Mobile Application —** a Flutter-based Android application operated by tea collectors in the field. It captures collection data, performs QR-based farmer identification, records field deductions and advances, and operates with complete functional independence from any network connection. It is the primary realisation of the offline-first constraint (NFR-17, FR-20).
- **Office Desktop / Web Application —** a React-based client operated by the Owner and Office Staff for farmer administration, deduction management, tea-rate configuration, payment calculation, payment approval, and reporting.
- **Backend Services —** a suite of independently deployable Spring Boot microservices (Authentication, Farmer, Route, Collection, Deduction, Payment, Bank Integration, SMS, Reporting, Audit, and Synchronisation) fronted by a Spring Cloud API Gateway. These services constitute the central source of truth and the integration surface for external systems.
- **Data Stores —** an embedded SQLite database on each mobile device (the local-first store), a PostgreSQL instance backing the office environment, and a central PostgreSQL cluster in the cloud environment.
- **External Integrations —** the Bank Transfer API (for disbursement of approved payments) and the SMS Gateway (for payment confirmation to farmers).

The following are explicitly outside the scope of this design: the internal implementation of third-party Bank and SMS providers; physical procurement of mobile and office hardware; and business-process change management. These are treated only at their integration boundaries.

1.3 References

#	Reference
1	IEEE 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions. The structure and content of this SDS follow the design-viewpoint model established by this standard.

#	Reference
2	ISO/IEC/IEEE 42010:2011 — Systems and software engineering — Architecture description. The architectural design (Section 3) is organised around architectural concerns and stakeholder viewpoints in accordance with this standard.
3	TeaRoutePay Software Requirements Specification, Group 09, document TCPS-SRS-2026-v1.0 — the controlling requirements baseline for this design.
4	SENG 31242 — System Design Project, Course Guideline & Industrial Standards, Version 1.0, Software Engineering Teaching Unit, University of Kelaniya (2026) — the SDS structural, diagramming, and reporting standards applied throughout.
5	OWASP Top 10 (2021) — the threat taxonomy used as the basis for the security-design mitigations in Section 7.
6	REST API Design Guidelines — the conventions governing the internal and external interface specifications in Section 6.

1.4 Overview of the Document

The remainder of this document is organised as follows. Section 2 (System Overview) describes the system at a conceptual level and establishes its relationship to the SRS, with particular attention to the hybrid local-first architecture that underpins rural offline operation. Section 3 (Architectural Design) justifies the selected microservices architecture, presents the high-level architecture, component, and deployment views, and specifies the technology stack. Section 4 (Detailed Design) presents the domain class model, the realisation of all twenty principal use cases through sequence diagrams, the state models for the system's stateful entities, and the complete database design including the data dictionary, data-integrity rules, and a requirements-traceability matrix. Section 5 (UI Design) documents the user-experience decisions and the navigation model. Section 6 (Interface Design) specifies the external and internal interfaces. Section 7 (Security Design) describes the authentication, authorisation, data-protection, auditability, and OWASP-aligned mitigation strategies. Section 8 (Non-Functional Design Decisions) consolidates the mapping of every non-functional requirement to the specific design mechanism that satisfies it. Architectural Decision Records are provided as an appendix.

Section	Title
1	Introduction
2	System Overview
3	Architectural Design
4	Detailed Design

Section	Title
5	User Interface Design
6	Interface Design
7	Security Design
8	Non-Functional Design Decisions
Appendix A	Architectural Decision Records

2. SYSTEM OVERVIEW

2.1 System Description and the Hybrid Local-First Architecture

TeaRoutePay is conceived as an offline-first, hybrid local-first distributed system. This characterisation is the single most consequential design decision in the document, and it is a direct response to the operating environment established by the SRS: collection is performed by lorry-borne collectors along rural routes where network connectivity is intermittent or absent, yet the business simultaneously requires a single, authoritative financial record of truth at the office and in the cloud.

The hybrid local-first model resolves this tension by maintaining authority at two tiers and reconciling them through controlled synchronisation:

- **The local tier** is realised by an embedded SQLite database on each collector's device. Every field transaction — a weight entry, a fertilizer distribution, a cash advance, or a field farmer registration — is committed first and unconditionally to local storage, with no dependency on connectivity. This guarantees that field operations never block on the network and that no field datum can be lost to a connectivity failure (addressing FR-05, FR-20, and NFR-17). At this tier, the device's local store is the temporary authority for records that have not yet been reconciled.
- **The central tier** is realised by a PostgreSQL database governed by the backend services. Once a record is synchronised — whether over the office LAN or over the Internet — the central tier becomes the permanent authority. Reconciliation is idempotent and conflict-aware (NFR-19), so that the same record synchronised twice does not create a duplicate, and competing edits to the same logical record are resolved deterministically by a last-write-wins policy keyed on device timestamp (FR-28).

The functional decomposition of the system into its principal modules, each of which is elaborated in the architectural and detailed designs that follow, is shown in Figure 1. The data flow from field collection to SMS confirmation — spanning the local tier, the office

tier, and the cloud tier — is depicted in Figure 2, which is reproduced exactly from the approved project workflow model.

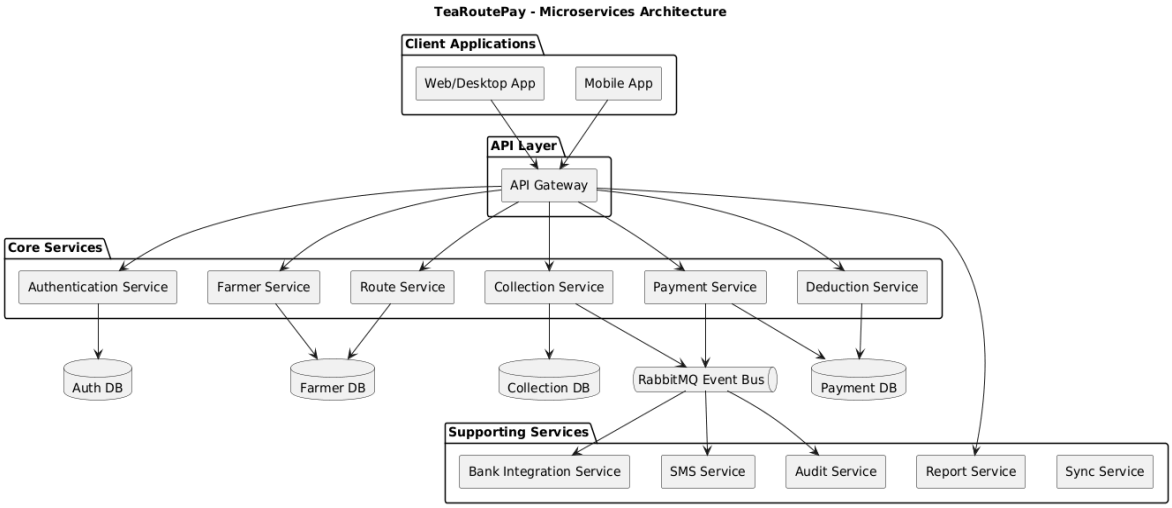

Figure 1: TeaRoutePay — High-Level System Workflow (Local, Cloud, and Payment Integration Pathways)

2.2 Relationship to the SRS

The design presented in this document implements the entirety of the SRS requirement baseline. Every functional requirement from FR-01 to FR-53 is realised by a service, a class, a use-case realisation, or a database structure described in Section 4, and every non-functional requirement from NFR-01 to NFR-29 is satisfied by a specific architectural or detailed-design mechanism consolidated in Section 8. The mapping is summarised at the level of quality attributes in the table below.

Requirement Category	Governing NFRs	Design Goal	Principal Design Mechanism
Performance	NFR-01 … NFR-05	Sub-second field interaction; fast payment computation and sync	Local SQLite reads; indexed queries; batch synchronisation; horizontal scaling
Security	NFR-06 … NFR-11	Encrypted communication and storage; immutable financial records	TLS 1.3; AES-256 at rest; RS256 JWT; immutable audit log; record locking
Usability	NFR-12 … NFR-16	Field usability for low-proficiency users; localisation	Large touch targets; offline indicators; Sinhala/Tamil/English UI
Reliability	NFR-17 … NFR-21	Zero field data loss; deterministic, idempotent processing	Write-ahead local persistence; ACID transactions; idempotent sync; retry with back-off

Requirement Category	Governing NFRs	Design Goal	Principal Design Mechanism
Scalability	NFR-22 … NFR-24	Growth in farmers, devices, and offices	Stateless services; load-balanced gateway; per-service databases
Maintainability	NFR-25 … NFR-29	Independent evolution and operability	Microservice isolation; schema migrations; externalised configuration; structured logging

3. ARCHITECTURAL DESIGN

3.1 Selected Architecture and Justification

The architecture selected for TeaRoutePay is a microservices architecture for the backend, deployed in concert with an offline-first mobile architecture on the collector device. This pairing is a deliberate response to the system's two dominant and partially opposing forces: the need for a robust, independently scalable, fault-isolated central platform that processes financial transactions of material value, and the need for a self-sufficient field client that continues to function with zero network dependency.

The microservices style decomposes the backend into a set of independently deployable services, each owning a bounded business capability and its own data store. The justification, expressed against the quality attributes prioritised by the SRS, is as follows.

Quality Attribute	Architectural Benefit	Requirements Addressed
Scalability	Each service scales independently behind the gateway; the computationally intensive Payment Service can be scaled at month-end without scaling unrelated services. The backend permits horizontal scaling without code change.	NFR-22, NFR-23, NFR-24
Fault isolation / Availability	A failure in a non-critical service (e.g., Reporting) cannot cascade into the collection or payment path. Combined with offline-first operation, the system tolerates partial outages gracefully.	NFR-17, NFR-21
Maintainability	Services can be modified, tested, and redeployed in isolation, supporting independent evolution of the codebase.	NFR-25, NFR-26, NFR-27
Security	Each service enforces capability-specific access control; the gateway provides a single, hardened security perimeter for authentication and rate-limiting.	NFR-06, NFR-08, FR-02
Deployability	Containerised services are deployed and orchestrated independently, enabling continuous delivery.	NFR-25

The offline-first mobile architecture treats local persistence as the primary write path and treats synchronisation as a deferred, asynchronous concern. This makes the field client immune to connectivity loss and is the direct architectural realisation of FR-20 and NFR-17.

Encryption of the local store at rest (SQLCipher) and idempotent reconciliation complete the design (NFR-07, NFR-19).

Alternatives considered and rejected: A monolithic architecture was rejected because its tight coupling impedes the independent scaling of the payment-calculation workload and complicates the integration of the offline synchronisation subsystem. A layered (n-tier) architecture was rejected because it does not naturally accommodate the distributed, independently deployable topology required to host field, office, and cloud tiers with differing availability and scaling profiles. These decisions are recorded formally in Appendix A.

3.2 High-Level Microservices Architecture

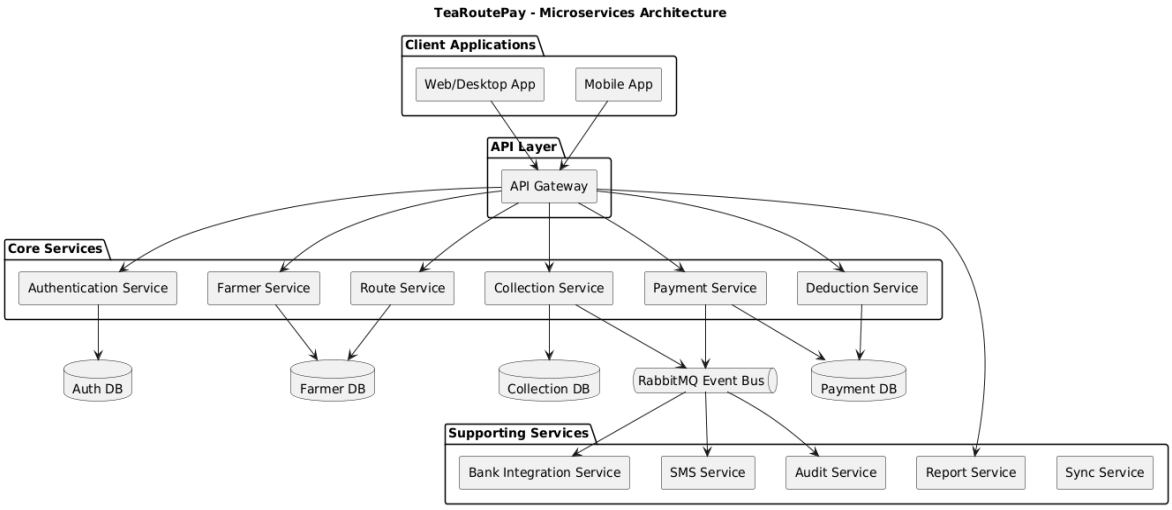

Figure 2: TeaRoutePay Microservices Architecture — Client, API, Core Service, and Supporting Service tiers

The high-level architecture organises the system into four horizontal tiers. At the top, the Client Applications tier comprises the Web/Desktop application (Office Staff and Owner) and the Mobile application (Collector). Neither client communicates with a service directly; all traffic is mediated by the API Layer, whose single ingress point is the API Gateway. The gateway is the architectural locus of cross-cutting concerns — request routing, authentication enforcement, and rate limiting — and it is the only component exposed to client networks. Concentrating these responsibilities at the gateway keeps the downstream

services free of duplicated security logic and provides a single, auditable security perimeter, directly supporting the access-control and session requirements (FR-01, FR-02, NFR-08).

The Core Services tier contains the six services that own the system's primary transactional capabilities: Authentication, Farmer, Route, Collection, Payment, and Deduction. The diagram shows the database-per-service pattern in operation: the Authentication Service owns Auth DB; the Farmer and Route services are backed by Farmer DB; the Collection Service owns Collection DB; and the Payment and Deduction services are backed by Payment DB. This per-service data ownership permits each service to be scaled, migrated, and secured independently, and is the structural basis for the scalability requirements (NFR-22 to NFR-24) and the schema-migration requirement (NFR-26).

The RabbitMQ Event Bus and the Supporting Services tier — Bank Integration, SMS, Audit, Report, and Sync — occupy the lower section of the diagram. The event bus decouples the synchronous payment path from the asynchronous side-effects that must follow a financial event. When the Payment Service completes and a payment run is approved, it publishes events to the bus rather than invoking downstream services in line; the Bank Integration, SMS, and Audit services consume those events independently. This decoupling allows external-integration latency and transient failures to be absorbed through retry and back-off without blocking the core financial transaction (NFR-21, FR-43 to FR-48). The Audit Service captures an immutable record of every financially significant event, which is the architectural foundation for NFR-10 and the auditability design of Section 7.

3.3 Service Responsibilities

Each backend service owns a single bounded capability. The responsibilities and data ownership of each are specified below.

Service	Responsibilities	Owns	Principal Requirements
Authentication Service	User login, JWT issuance and validation, user-account management, role-based access control	Auth DB (Users, Roles, Permissions, RefreshTokens)	FR-01, FR-02, FR-03, FR-04, FR-05, FR-06

Service	Responsibilities	Owns	Principal Requirements
Farmer Service	Farmer registration, QR-code generation, profile update and deactivation, farmer search	Farmer DB (Farmers, QRMappings, FarmerAudit)	FR-07 to FR-13
Route Service	Route creation, farmer-to-route assignment, collector allocation, scheduling	Route DB	FR-14, FR-15, FR-16
Collection Service	Weight-record ingestion, offline synchronisation reconciliation, duplicate detection	Collection DB (Collections, SyncRecords)	FR-17 to FR-29
Deduction Service	Loan, fertilizer, and custom deduction recording; installment scheduling	Deduction DB	FR-32 to FR-36
Payment Service	Tea-rate management, monthly gross/net calculation, payment preview, approval, record locking	Payment DB	FR-30, FR-31, FR-37 to FR-42
Bank Integration Service	Bank Transfer API communication, retry logic, transfer-status retrieval, re-submission	— (integrates via gateway)	FR-43 to FR-46
SMS Service	SMS composition, localisation, delivery tracking	—	FR-47, FR-48, FR-49
Reporting Service	PDF and CSV report generation and export	— (reads from owning services)	FR-50, FR-51, FR-52, FR-53
Audit Service	Immutable, append-only audit logging; compliance tracking	Audit store	FR-06, FR-42, FR-45, NFR-10
Sync Service	Coordination of LAN and cloud synchronisation sessions; conflict logging	—	FR-25 to FR-29

3.4 Component Diagram

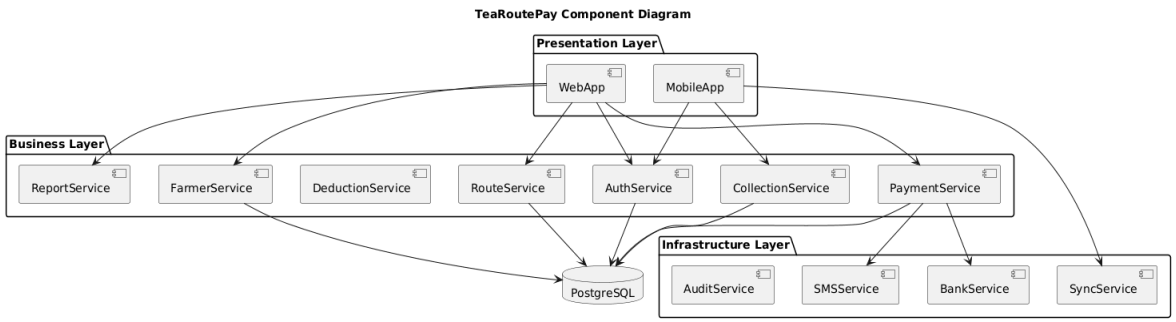

Figure 3: TeaRoutePay Component Diagram — Presentation, Business, and Infrastructure layers with dependency direction

The component diagram presents the system organised by logical layer, making the dependency direction explicit. The Presentation Layer contains the WebApp and MobileApp components. The Business Layer contains the service components — ReportService, FarmerService, DeductionService, RouteService, AuthService, CollectionService, and PaymentService — each a self-contained unit of business logic. The Infrastructure Layer contains the cross-cutting and integration components: AuditService, SMSService, BankService, and SyncService.

The dependency arrows encode an important design rule: presentation components depend on business components, and business components depend downward on shared infrastructure, but no business component depends on another business component for its core transaction. This acyclic, downward-only dependency structure makes the services independently testable and deployable, and prevents the emergence of the tangled coupling that motivated the rejection of the monolithic alternative. The convergence of multiple services on the PostgreSQL component in this view is a logical convergence on the relational technology; at deployment time each service is bound to its own schema, preserving the database-per-service property.

3.5 Deployment Diagram

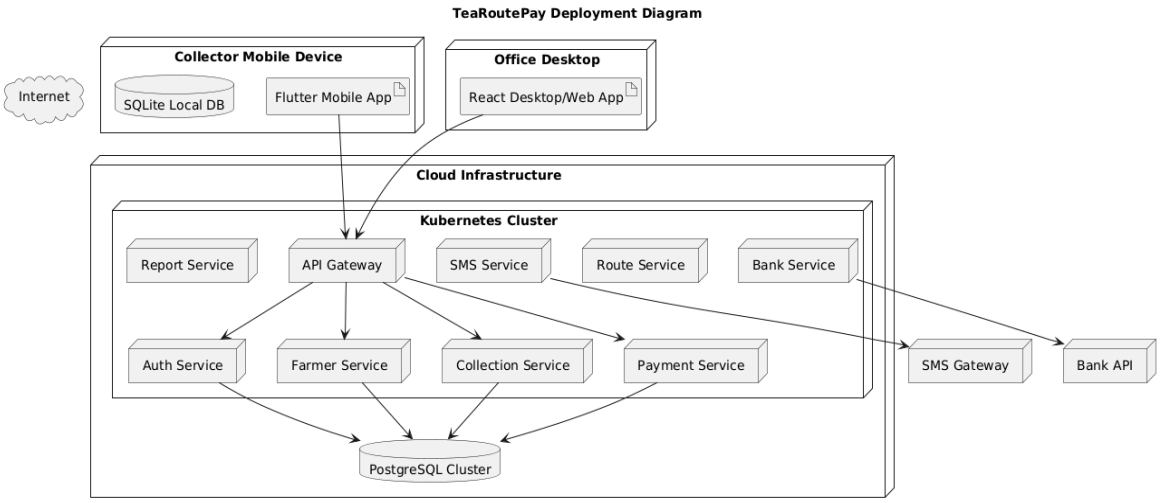

Figure 4: TeaRoutePay Deployment Diagram — Mobile Device, Office Desktop, and Kubernetes Cloud Infrastructure nodes

The deployment diagram fixes the physical placement of every executable component. Three deployment nodes are shown. The Collector Mobile Device node hosts the Flutter Mobile App and its co-resident SQLite Local DB; their co-residence is the physical embodiment of the offline-first principle — the field client reads and writes its authoritative working set entirely on-device, delivering the sub-second interaction of NFR-01 and the zero-data-loss guarantee of NFR-17. The Office Desktop node hosts the React Desktop/Web App used by Office Staff and the Owner.

The Cloud Infrastructure node contains a Kubernetes Cluster that orchestrates the containerised services — API Gateway, Auth Service, Farmer Service, Collection Service, Payment Service, Route Service, SMS Service, Bank Service, and Report Service — together with the backing PostgreSQL Cluster. Kubernetes provides the auto-scaling and self-healing that satisfy the horizontal-scaling requirement (NFR-24). The diagram also shows the two external integration endpoints, SMS Gateway and Bank API, reached outbound from the cluster. The combination of per-service schema ownership (described in Section 3.3) and a single managed database platform achieves a pragmatic balance between operational simplicity and architectural independence.

3.6 Technology Stack

The technology stack is specified below. Each selection is justified against the requirement or quality attribute it serves; collectively these choices constitute the implementation contract that the development phase must honour.

Component	Technology	Technical Justification	Requirements Addressed
Mobile App	Flutter	A single Dart codebase compiles to native Android, giving native-grade camera access for QR scanning and a high-performance offline UI. Flutter's reactive widget model supports the large touch targets and offline indicators required for low-proficiency field users.	FR-17 to FR-24, NFR-12, NFR-13
Office Client	React.js	A component-based, browser-deployable client that runs as both a desktop-hosted and web application, supporting tabular, dashboard-oriented administrative workflows.	FR-30 to FR-53, NFR-16

Component	Technology	Technical Justification	Requirements Addressed
API Gateway	Spring Cloud Gateway	Provides a single, hardened ingress with built-in routing, authentication enforcement, and rate limiting, concentrating the security perimeter at one auditable point.	FR-01, FR-02, NFR-06, NFR-08
Backend Services	Spring Boot	The industry standard for enterprise, security-critical financial APIs; strong typing and declarative @Transactional boundaries provide the ACID guarantees required for monetary processing.	FR-37, FR-38, NFR-18, NFR-20
Central Database	PostgreSQL	A production-grade relational engine providing full ACID compliance, the correctness foundation for financial records, and per-service schema isolation.	NFR-18, NFR-22
Mobile Local Store	SQLite (SQLCipher)	An embedded, zero-administration, transactional store that delivers on-device durability and, through SQLCipher, AES-256 encryption of data at rest on the device.	FR-05, FR-20, NFR-07, NFR-17
Message Broker	RabbitMQ	Decouples the synchronous payment path from asynchronous side-effects (bank transfer, SMS, audit), enabling reliable, retryable, event-driven processing.	FR-43 to FR-48, NFR-10, NFR-21
Authentication	JWT (RS256) + RBAC	Stateless, signed tokens permit scalable authentication across services without server-side session affinity; RS256 asymmetric signing allows services to verify tokens without holding the signing key.	FR-01 to FR-04, NFR-08
Containerisation	Docker	Produces immutable, reproducible service images, guaranteeing parity between environments.	NFR-25
Orchestration	Kubernetes	Provides declarative deployment, horizontal auto-scaling, and self-healing for the service fleet.	NFR-24
Reporting	JasperReports	A Java-native engine integrating directly with Spring Boot to produce structured PDF reports and receipts.	FR-50 to FR-53
CI/CD	GitHub Actions	Automates build, test, and deployment, enforcing the coverage and quality gates of the maintainability requirements.	NFR-25, NFR-28

4. DETAILED DESIGN

4.1 Domain Class Diagram

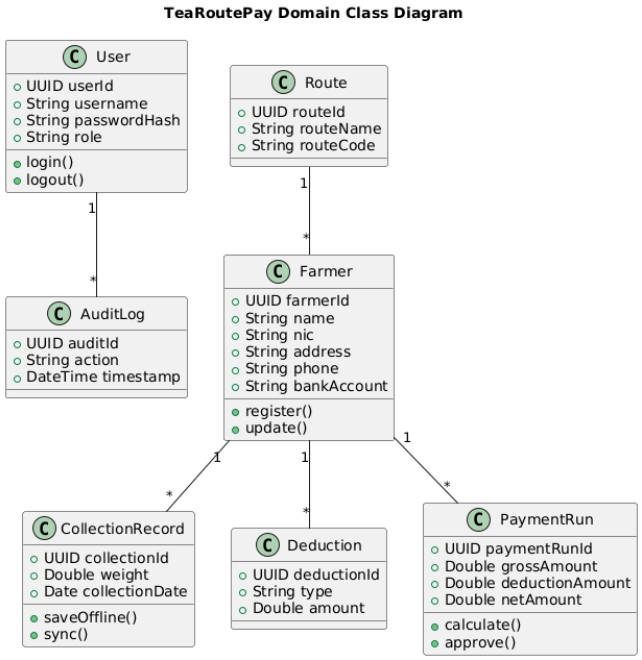

Figure 5: TeaRoutePay Domain Class Diagram — core entities, attributes, methods, and associations

The domain class diagram defines the core entities of the TeaRoutePay business domain, their attributes, their behaviour, and the associations and multiplicities that bind them. The model is intentionally aligned with the database design of Section 4.4, so that each persistent class corresponds to a table and each association corresponds to a foreign-key relationship.

User. This class represents any authenticated principal and carries userId : UUID, username : String, passwordHash : String, and role : String. Its behaviour — login() and logout() — encapsulates the authentication lifecycle. The role attribute is the discriminator on which all role-based access control decisions are made (FR-02). The one-to-many association from User to AuditLog means every audit entry is attributable to exactly one acting user, providing the structural guarantee behind the auditability requirement (NFR-10).

Farmer. The Farmer class is the pivot of the domain. It carries farmerId : UUID, name, nic, address, phone, and bankAccount, and exposes register() and update() behaviour. A Route aggregates many Farmers (1-to-*), each Farmer is the subject of many CollectionRecords

(1-to-*), many Deductions (1-to-*), and many PaymentRun outcomes (1-to-*). The uniqueness of nic (enforced per FR-08) and the immutability of a farmer with payment history (FR-12) are realised against this class.

CollectionRecord. This class models a single field weight entry. It carries collectionId : UUID, weight : Double, and collectionDate : Date, and exposes the offline-first behaviour saveOffline() and sync(). The lifecycle of this class — from local creation, through the pending-sync and synced states, to its final immutable locking once it has contributed to an approved payment — is modelled explicitly by the Collection Record state machine in Section 4.3.

PaymentRun. This class represents one month's payment computation for the business. It carries paymentRunId : UUID, grossAmount, deductionAmount, and netAmount, and exposes calculate() and approve(). The transition of a PaymentRun from a mutable draft to a locked, immutable, approved artefact (FR-41) is the most safety-critical state transition in the system and is modelled by the Payment Run state machine in Section 4.3.

All monetary attributes shown as Double in the class diagram are, per the financial-accuracy constraint of the SRS, implemented as fixed-point (integer-cent) representations in storage and computation; floating-point arithmetic is prohibited for monetary values (FR-37, NFR-20). The class diagram captures the logical attribute; Section 4.4 specifies its precise persistent type.

4.2 Sequence Diagrams (Use-Case Realisations)

Each of the twenty principal use cases of the SRS is realised below by a sequence diagram reproduced exactly from the approved design. For each, the step-by-step technical execution is described with particular attention to data validation and to the state changes that the interaction effects. The diagrams share a consistent participant convention: an actor initiates an interaction with a client application (Mobile or Web), which dispatches through the API Gateway to the owning service, which reads or writes its database, a cache, or an external API.

SD-01 — Log in to Mobile App (UC-01)

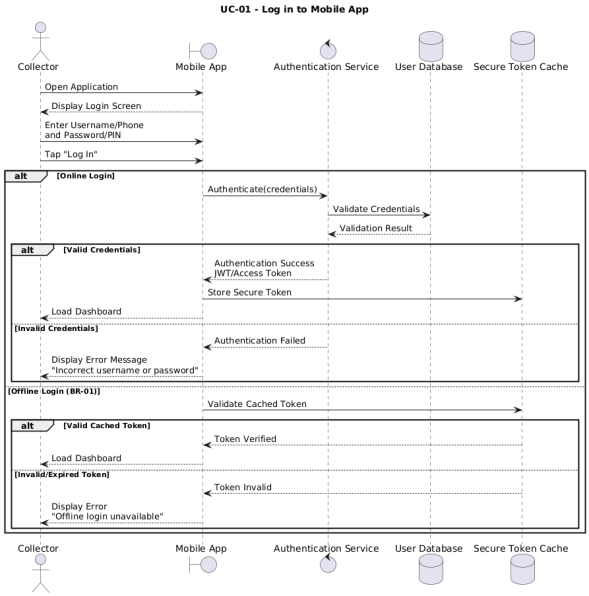

Figure 6: SD-01 — Log in to Mobile App (UC-01)

The Collector opens the application and is presented with the login screen. In the online branch, the Mobile App forwards credentials to the Authentication Service, which validates them against the User Database. On success, a JWT access token is returned, stored in the Secure Token Cache, and the dashboard is loaded. On invalid credentials, a neutral error message is displayed without disclosing which field was wrong. In the offline branch (BR-01), a previously cached token is validated locally: a verified token loads the dashboard, while an invalid or expired token yields an 'Offline login unavailable' message. The state change is the establishment of an authenticated, role-bound session on which all subsequent authorisation depends (FR-01, FR-02, NFR-08).

SD-02 — View Assigned Route (UC-02)

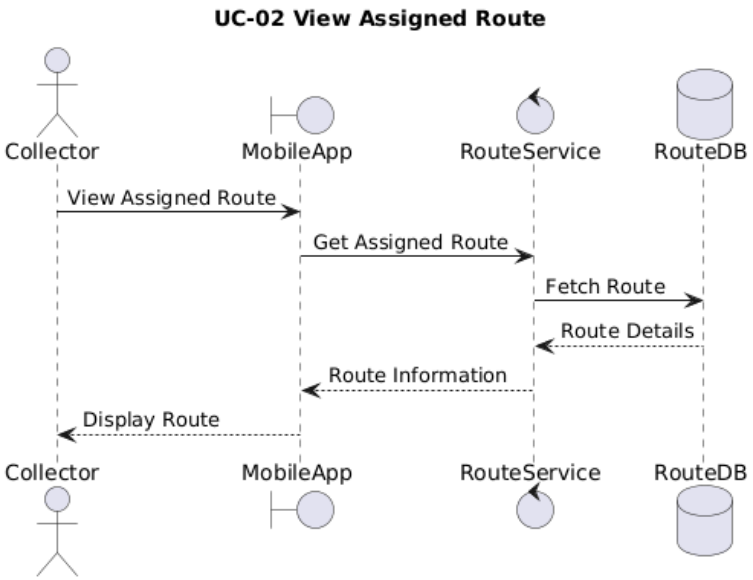

Figure 7: SD-02 — View Assigned Route (UC-02)

Following authentication, the Collector requests their assigned route for the session. The Mobile App resolves the request against the locally synchronised route and farmer working set, returning the ordered list of farmers assigned to the collector's active route. Because the route working set is pre-synchronised to the device, this interaction completes entirely on-device with no connectivity dependency, allowing route selection to begin a collection session in the field (FR-17). No persistent state change occurs.

SD-03 — Scan Farmer QR Code (UC-03)

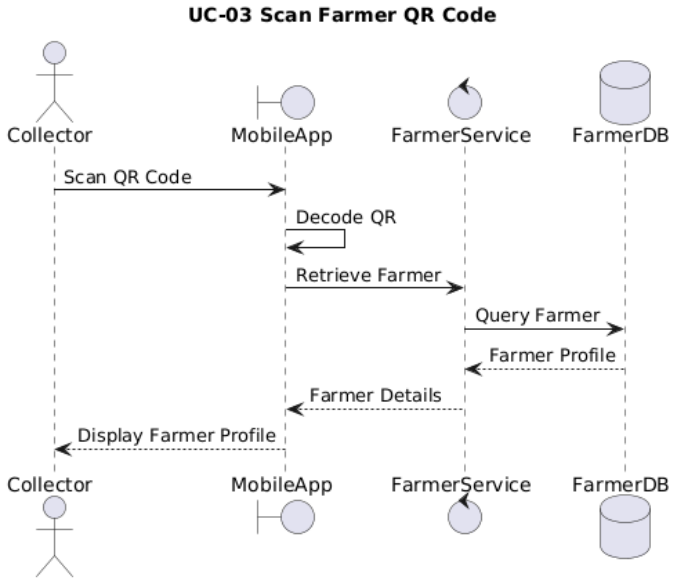

Figure 8: SD-03 — Scan Farmer QR Code (UC-03)

The Collector invokes the device camera to scan the farmer's QR card. The Mobile App decodes the encoded farmer UUID and resolves it against the local farmer cache to load and display the farmer's identifying details for confirmation. Validation ensures the decoded UUID corresponds to an active farmer on the current route; an unresolved or out-of-route code is rejected with corrective guidance, and the manual-fallback path is offered (FR-19). The QR payload encodes only the farmer UUID — never personal or financial data — bounding the consequences of a lost card (FR-09, FR-18).

SD-04 — Record Leaf Weight (UC-04)

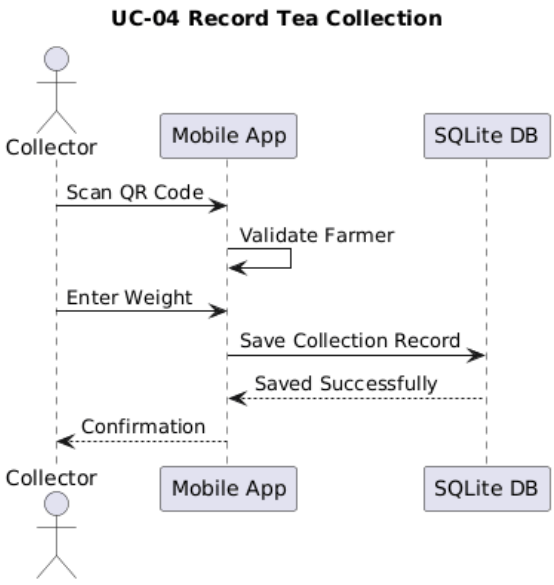

Figure 9: SD-04 — Record Leaf Weight (UC-04)

With a farmer identified, the Collector enters the leaf weight. The Mobile App validates the entry and commits the record to the local SQLite store immediately with a PendingSync status, independent of any network state. This unconditional local write is the defining moment of the offline-first design: the record is durable the instant it is saved, satisfying FR-20 and the zero-loss guarantee of NFR-17, and the running session total is updated (FR-23). The duplicate-prevention rule (FR-22) is evaluated here: a second entry for the same farmer in the same session prompts explicit confirmation. The state change is the creation of a new CollectionRecord in the PendingSync state.

SD-05 — Item Distribution and Record Keeping (UC-05)

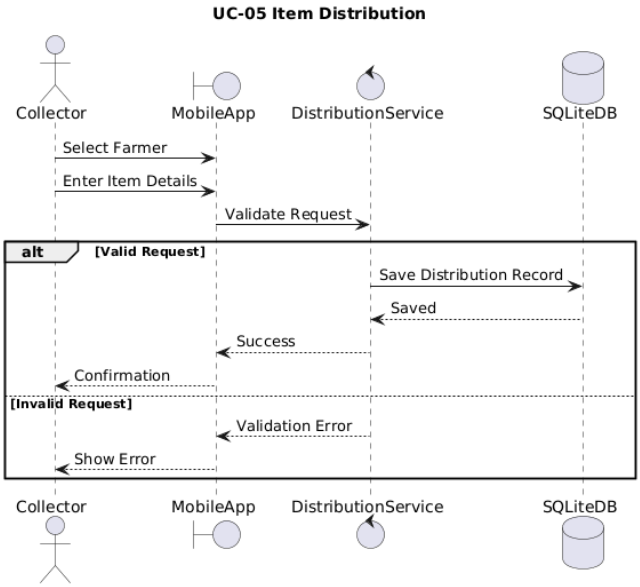

Figure 10: SD-05 — Item Distribution and Record Keeping (UC-05)

When fertilizer or other input materials are distributed to a farmer in the field, the Collector records the distribution through a quick-select flow. The Mobile App writes the distribution as a field-originated deduction record to local storage with PendingSync status, mirroring the weight-entry persistence pattern. On synchronisation, the Deduction Service ingests the record and schedules it as a deduction against the farmer's account (FR-34), to be applied during the monthly calculation. The interaction is fully offline-capable, and the state change is the creation of a pending field-deduction record.

SD-06 — Issue Advance Payment and Record Keeping (UC-06)

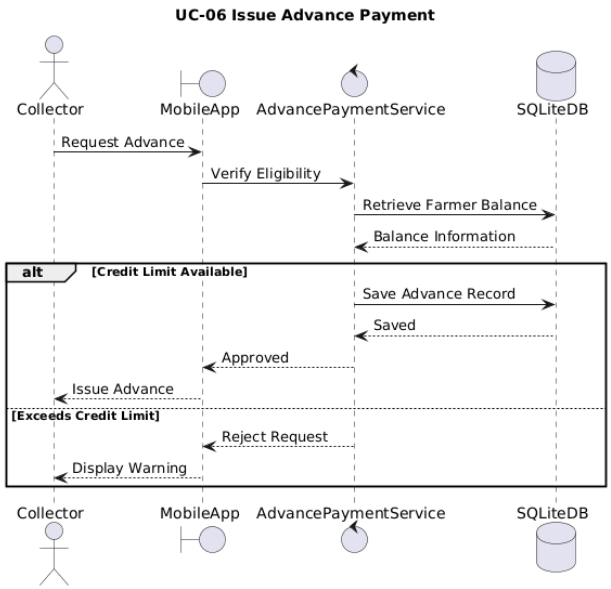

Figure 11: SD-06 — Issue Advance Payment and Record Keeping (UC-06)

The Collector issues a cash advance to a farmer. Before approving the transaction, the Mobile App queries the local database to verify the farmer's accumulated balance and any applicable credit limit. A request within the limit is recorded locally as a pending advance and the physical handover proceeds; a request exceeding the limit follows the exception path, which requires an Owner override review before completion. The advance is reconciled to the Deduction Service on sync, where it joins the set of liabilities applied at month-end. This realisation enforces a business-rule validation at the point of field action while preserving the offline-first persistence guarantee.

SD-07 — Synchronise Collection Data (UC-07)

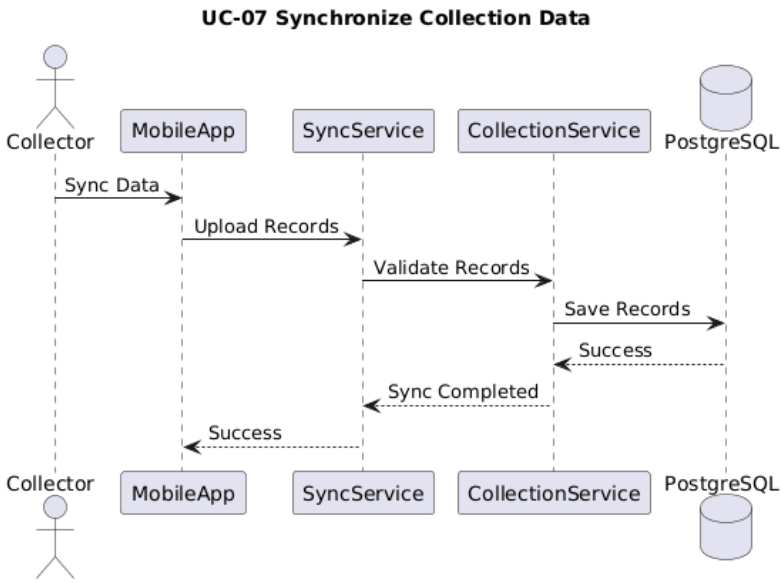

Figure 12: SD-07 — Synchronise Collection Data (UC-07)

At the end of a route, the Collector triggers synchronisation. The system selects the configured pathway (Local LAN, Cloud, or Local-First-Cloud-Backup per FR-25) and transmits all PendingSync records in a batch. The receiving tier validates and persists the records, returns a confirmation, and the Mobile App marks the corresponding local records as Synced. The reconciliation is idempotent — a record already present is acknowledged but not re-inserted (NFR-19) — and conflicts are resolved by last-write-wins on device timestamp and logged for Owner review (FR-28). Failed transmissions are retried up to three times with exponential back-off before the user is notified (FR-29, NFR-21). The state change is the transition of each CollectionRecord from PendingSync to Synced.

SD-08 — Log in to Desktop App (UC-08)

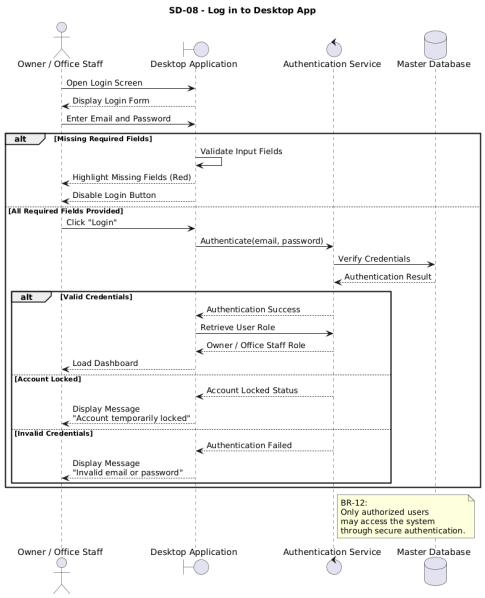

Figure 13: SD-08 — Log in to Desktop App (UC-08)

The Office Staff or Owner authenticates through the Web/Desktop client. Credentials are dispatched through the gateway to the Authentication Service and validated; on success a signed JWT is returned and held for the session, and the role-appropriate dashboard is loaded. Unlike the mobile flow, the desktop flow has no offline branch, as office operation presupposes connectivity to the central tier. Account lockout after five consecutive failures (FR-03) and the configurable idle session timeout (FR-04) are enforced on this path.

SD-09 — Register Farmer (UC-09)

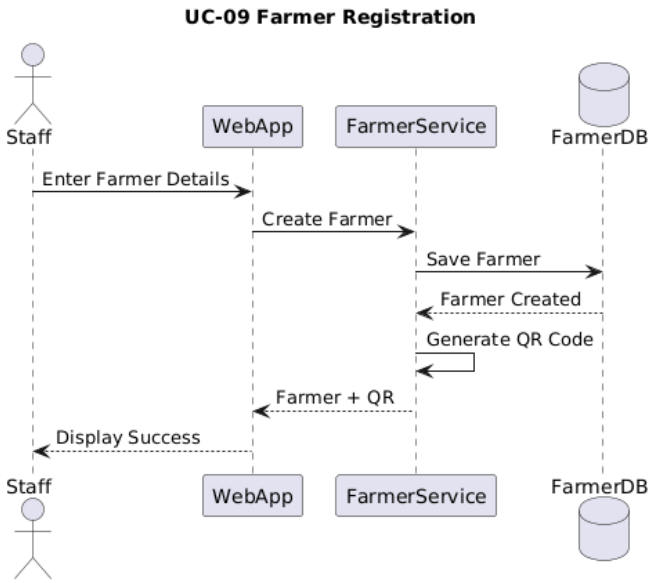

Figure 14: SD-09 — Register Farmer (UC-09)

Office Staff (or a Collector in the field) enters the mandatory farmer attributes — name, NIC, address, phone, route assignment, and bank account. The Farmer Service validates the submission, with the critical validation being the uniqueness of the NIC across all records, including deactivated ones (FR-08); a duplicate is rejected. On success, the farmer is persisted and a unique QR code encoding the new farmer UUID is generated (FR-09). The creation is written to the farmer audit trail with the acting user and timestamp (FR-11). The state change is the creation of a Farmer in the Registered state and the minting of its QR identity.

SD-10 — Manage Farmer Profiles (UC-10)

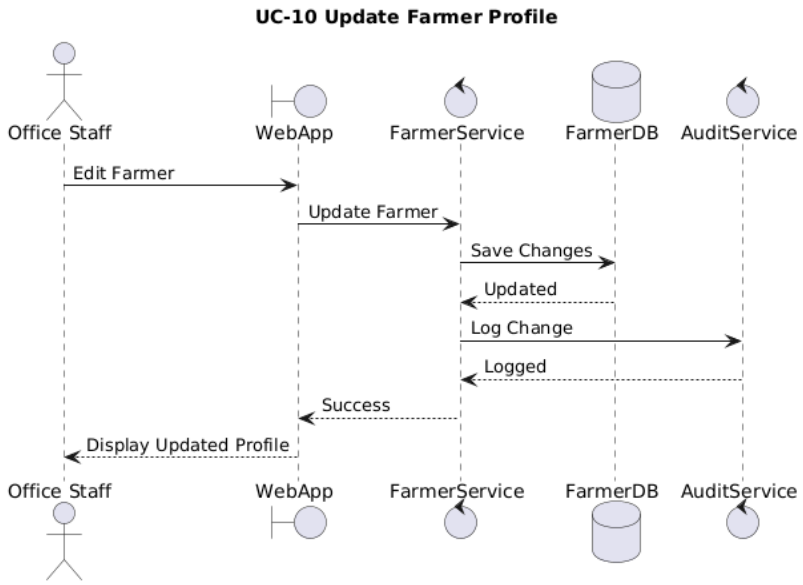

Figure 15: SD-10 — Manage Farmer Profiles (UC-10)

Office Staff retrieves a farmer profile and updates editable fields, or deactivates the farmer. The Farmer Service applies the change transactionally and appends an audit entry capturing the performing user's identity, timestamp, and before/after values (FR-11). Deactivation is a soft delete: a farmer carrying collection or payment history cannot be hard-deleted (FR-12), so the operation transitions the Farmer to the Deactivated state rather than removing the row, preserving the historical integrity on which past payment runs depend.

SD-11 — Manage Deductions (UC-11)

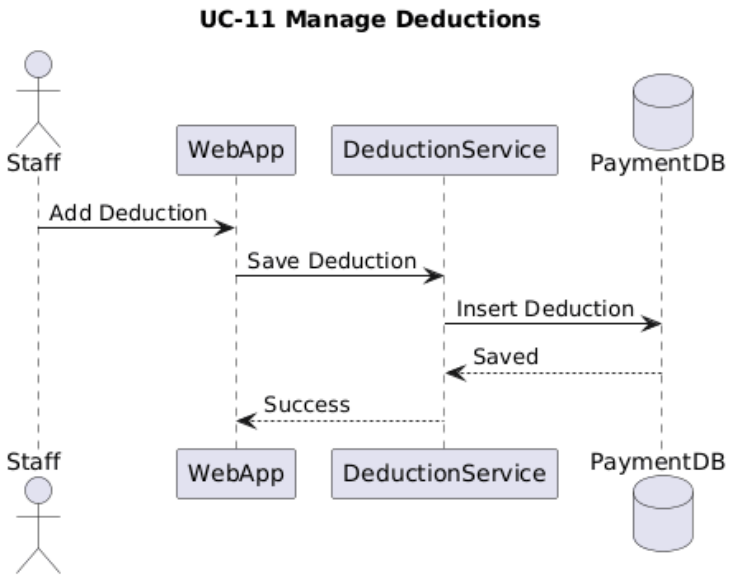

Figure 16: SD-11 — Manage Deductions (UC-11)

Office Staff records a liability against a farmer — a loan (with total amount, monthly installment, start month, and description), a fertilizer deduction, or a custom deduction type. The Deduction Service validates the parameters and persists the deduction, scheduling recurring installments where applicable (FR-32, FR-33, FR-34, FR-35). Field-originated deductions synced from the mobile app are surfaced here for review and confirmation before they participate in a calculation. The state change is the creation or amendment of scheduled liabilities that the Payment Service will consume at month-end.

SD-12 — Set Monthly Tea Rate (UC-12)

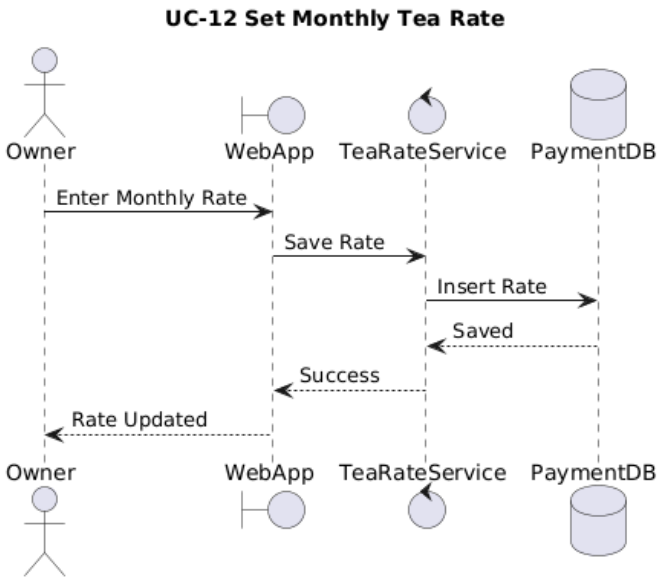

Figure 17: SD-12 — Set Monthly Tea Rate (UC-12)

The Owner or Office Staff sets the purchase rate (LKR per kilogram) for a given calendar month. The Payment Service validates that the target month does not already have a completed payment calculation — a rate for such a month is immutable (FR-31) — and persists the new rate to the rate history. The state change is the establishment of the multiplier that the gross-payment calculation (FR-37) will apply for that month; because rates are historised, recomputation of any past month uses that month's rate, never the current one.

SD-13 — Calculate Monthly Payment (UC-13)

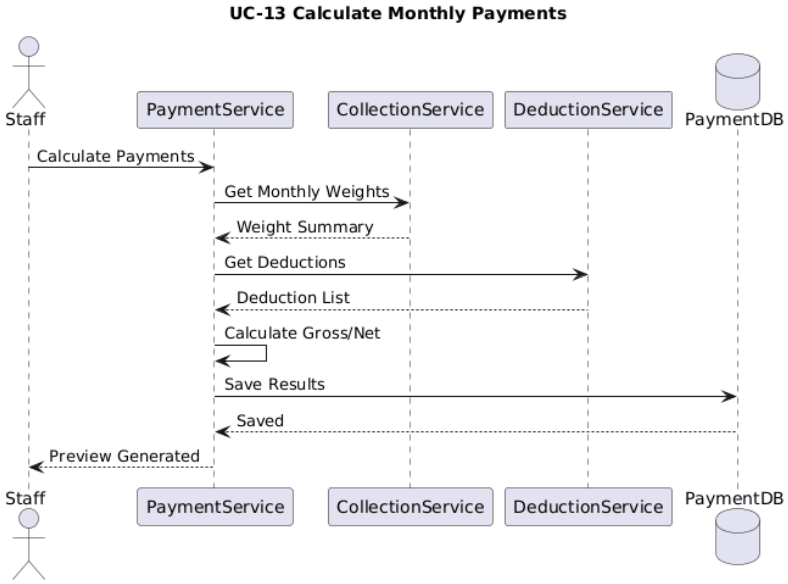

Figure 18: SD-13 — Calculate Monthly Payment (UC-13)

Office Staff initiates the monthly calculation. The Payment Service orchestrates the computation: it requests the monthly weight summary per farmer from the Collection Service, requests the applicable deduction list from the Deduction Service, and then computes, per farmer, the gross payment (total kilograms times the month's tea rate, in fixed-point arithmetic per FR-37) and the net payment (gross minus the sum of applicable deductions, per FR-38). Results are persisted and a preview is returned. A farmer whose net would be negative triggers the negative-payment alert and blocks the run from advancing to approval until resolved (FR-36). The calculation is deterministic (NFR-20) and completes within the performance envelope of NFR-04. The state change is the transition of the PaymentRun from Draft to Calculated.

SD-14 — View Payment Summary (UC-14)

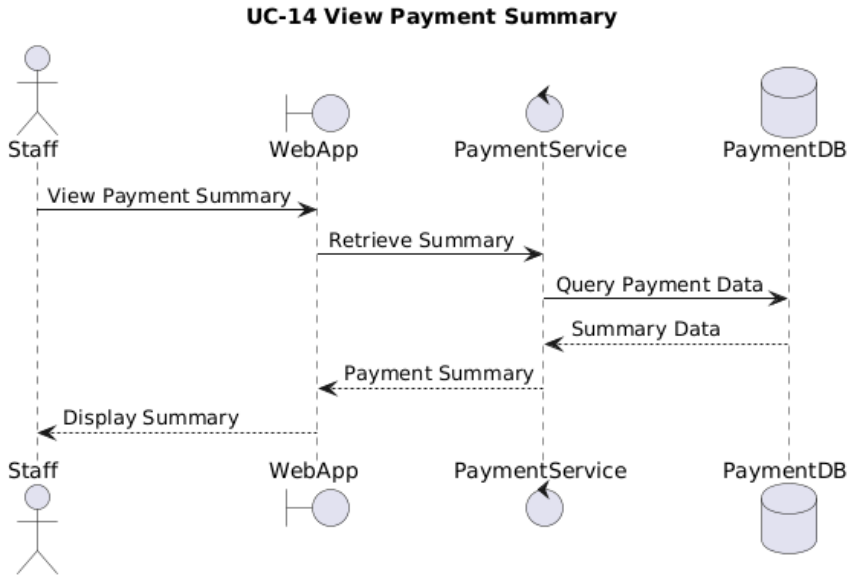

Figure 19: SD-14 — View Payment Summary (UC-14)

The Owner or Office Staff retrieves the computed preview, which presents for each farmer: total kilograms, gross payment, itemised deductions, and net payment in LKR (FR-39). The Payment Service serves the persisted results; the desktop client renders the tabular preview, loading within the performance bound of NFR-02. No state change occurs — this is a read interaction whose purpose is to furnish the Owner with the complete basis for the approval decision that follows.

SD-15 — Approve Payment Run (UC-15)

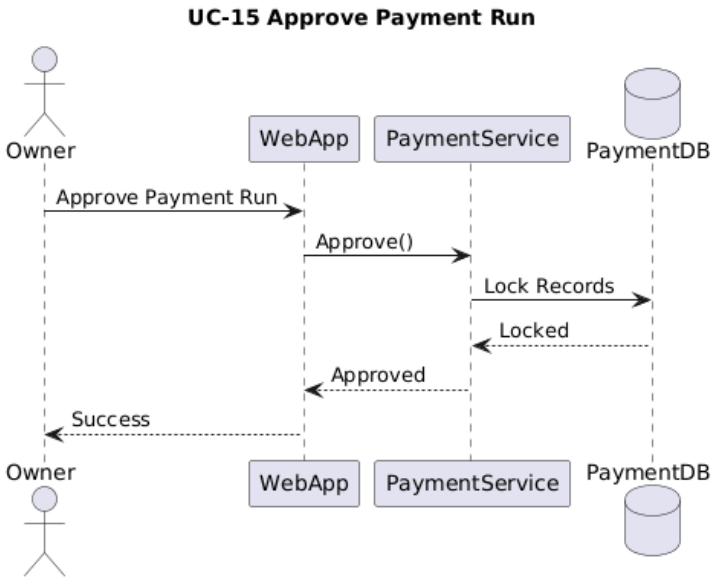

Figure 20: SD-15 — Approve Payment Run (UC-15)

The Owner reviews the preview and approves or rejects the run; a rejection must carry a reason (FR-40). On approval, the Payment Service performs the most safety-critical state transition in the system: every payment record in the run is locked and rendered immutable, after which no user may edit it (FR-41). The approving Owner's identity, approval timestamp, month, total approved amount, and any remarks are written to the immutable audit log (FR-42). Approval transitions the PaymentRun from PendingApproval to Approved and emits the event that initiates downstream disbursement. This separation of calculation (Staff) from approval (Owner) is the maker-checker control.

SD-16 — Process Bank Transfer (UC-16)

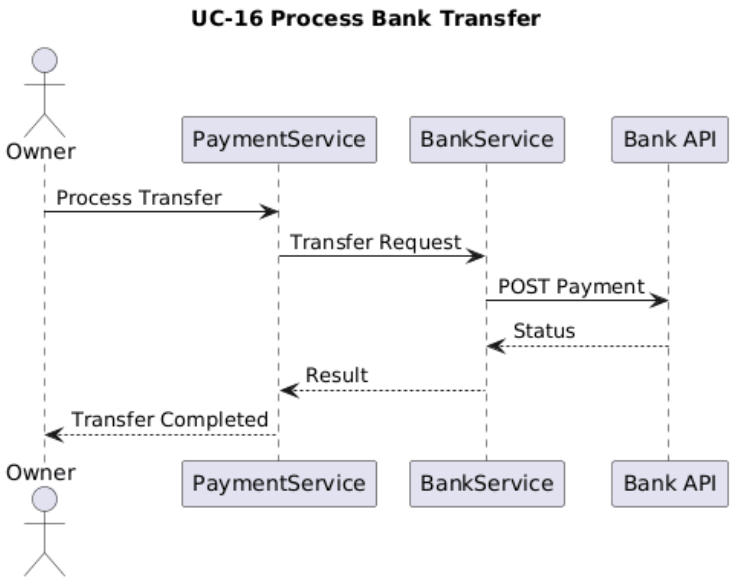

Figure 21: SD-16 — Process Bank Transfer (UC-16)

Upon approval, the Bank Integration Service submits the approved payments to the external Bank Transfer API, targeting submission within five minutes of approval (FR-43). For each payment it retrieves and records the transfer status — Success, Pending, or Failed (FR-44) — and writes the full request, response, HTTP status, and timestamp to the audit log (FR-45). Transient failures are retried with exponential back-off (max three attempts, NFR-21); a persistently failed transfer may be manually re-submitted once the cause is resolved (FR-46). The state change is the transition of the PaymentRun into BankProcessing and, on confirmation, the recording of a definitive per-payment transfer outcome.

SD-17 — Send SMS Notification (UC-17)

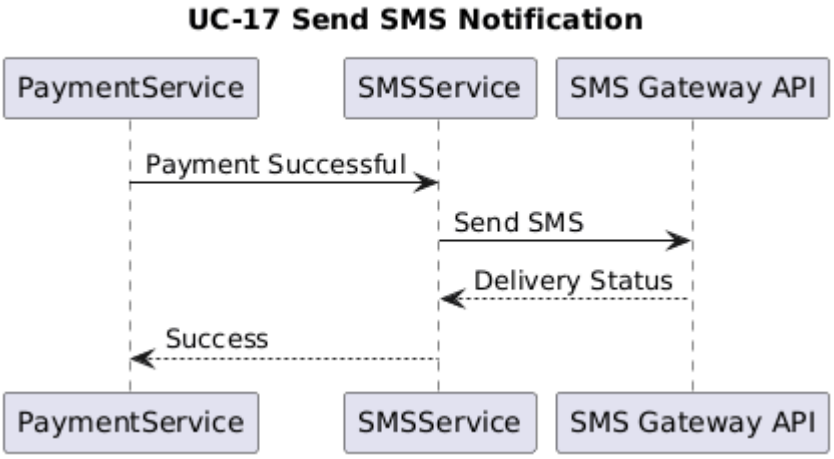

Figure 22: SD-17 — Send SMS Notification (UC-17)

Strictly after a bank transfer is confirmed successful — never at calculation or approval (FR-48, BR-25) — the SMS Service composes and dispatches a confirmation to the farmer stating the net amount in LKR and the payment month (FR-47). The message is rendered in the farmer's preferred language among Sinhala, Tamil, and English (FR-49). Delivery is tracked, and the loop with the farmer — the final node of the high-level workflow — is closed. The state change is the recording of a delivered (or failed-and-retryable) notification against the payment.

SD-18 — Generate Reports (UC-18)

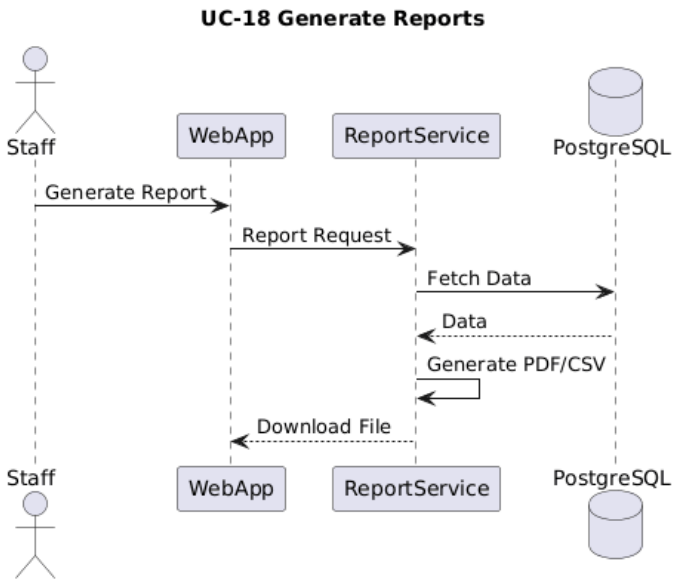

Figure 23: SD-18 — Generate Reports (UC-18)

An Owner or Office Staff requests a report — the daily collection report (FR-50) or the monthly payment report (FR-51). The Reporting Service aggregates the relevant data from the owning services and renders the output through JasperReports, offering export in both PDF and CSV (FR-52). Because reporting reads from rather than writes to the transactional services, report generation cannot perturb financial state. No state change occurs.

SD-19 — Manage User Accounts (UC-19)

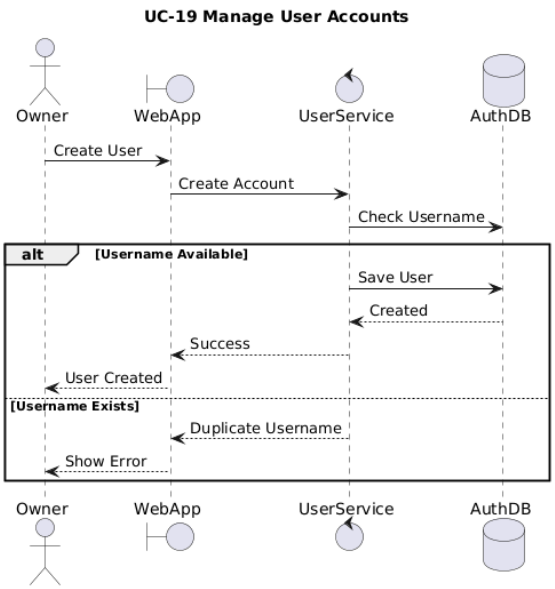

Figure 24: SD-19 — Manage User Accounts (UC-19)

The Owner manages system users: creating a Collector or Staff account, assigning a role and a default password, or deactivating an account (FR-05). The Authentication Service validates the submission — rejecting usernames that are not unique (BR-27) and passwords under eight characters — and persists the credential set. Every such administrative action is audit-logged (FR-06). The state change is the creation, modification, or deactivation of a principal and its associated role binding, which immediately governs that principal's subsequent access.

SD-20 — Manage Collection Routes (UC-20)

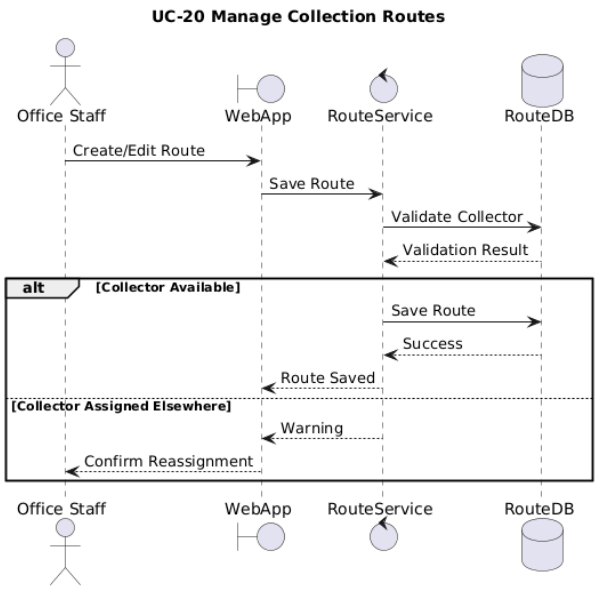

Figure 25: SD-20 — Manage Collection Routes (UC-20)

The Owner or Office Staff creates or edits a route, sets its name and code, and assigns a responsible Collector (FR-14); farmers are assigned or re-assigned to routes, with the full assignment history retained (FR-15). The Route Service enforces the governing business rules: a farmer may belong to only one active route at a time (BR-28), a route must exist before it can populate the farmer-registration dropdown (BR-29), and re-assigning an already-allocated collector raises a warning prompt. The state change is the creation or amendment of the route topology against which all field collection is organised.

4.3 State Machine Diagrams

The system contains three entities whose behaviour is governed by an explicit lifecycle. Modelling these as state machines makes the permissible transitions — and equally importantly the forbidden ones — unambiguous for implementation and testing.

4.3.1 Farmer State Machine

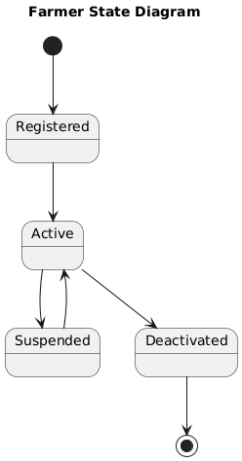

Figure 27: Farmer State Diagram — Registered, Active, Suspended, Deactivated lifecycle

A Farmer enters the Registered state upon creation (FR-07). From Registered it transitions to Active, the normal operating state in which the farmer appears in route collection lists and participates in payment runs. From Active, two transitions are possible: to Suspended, a reversible state (shown by the bidirectional transition) used for temporary holds, from which the farmer may be reactivated; and to Deactivated, the terminal soft-deleted state reached when a farmer ceases supply. The transition into Deactivated is the soft-delete of FR-12: the record and its history are retained, but the farmer no longer appears in active collection lists.

4.3.2 Payment Run State Machine

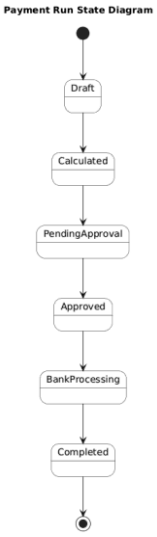

Figure 28: Payment Run State Diagram — Draft through Completed (irreversible linear lifecycle)

The PaymentRun lifecycle is strictly linear and irreversible — appropriate for a financial artefact. A run begins in Draft, transitions to Calculated when the gross/net computation

completes (FR-37, FR-38), and to PendingApproval when the preview is submitted for the Owner's decision (FR-39). Owner approval transitions it to Approved, at which point all constituent records are locked and immutable (FR-41) — this is the point of no return. From Approved the run enters BankProcessing as disbursement is executed (FR-43), and finally reaches Completed once transfers are confirmed, after which SMS notifications are emitted (FR-47). The absence of any backward transition from Approved onward is the structural enforcement of payment immutability; correction of an error in an approved run is handled not by mutation but by a compensating run.

4.3.3 Collection Record State Machine

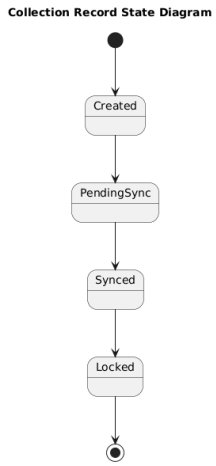

Figure 29: Collection Record State Diagram — Created, PendingSync, Synced, Locked

A CollectionRecord is Created the moment a weight is entered on the device, and immediately enters PendingSync — it is durable locally before it is ever transmitted (FR-20, NFR-17). On successful synchronisation it transitions to Synced, at point central authority is established (FR-26/FR-27). Once the record has contributed to an approved payment run, it transitions to Locked and becomes immutable, mirroring the immutability of the payment that consumed it (FR-41). This lifecycle is the per-record complement to the Payment Run lifecycle: together they guarantee that the chain from a field weight entry to a disbursed payment is, once approved, tamper-evident end to end.

4.4 Database Design

4.4.1 Central (Cloud / Office) Entity-Relationship Model

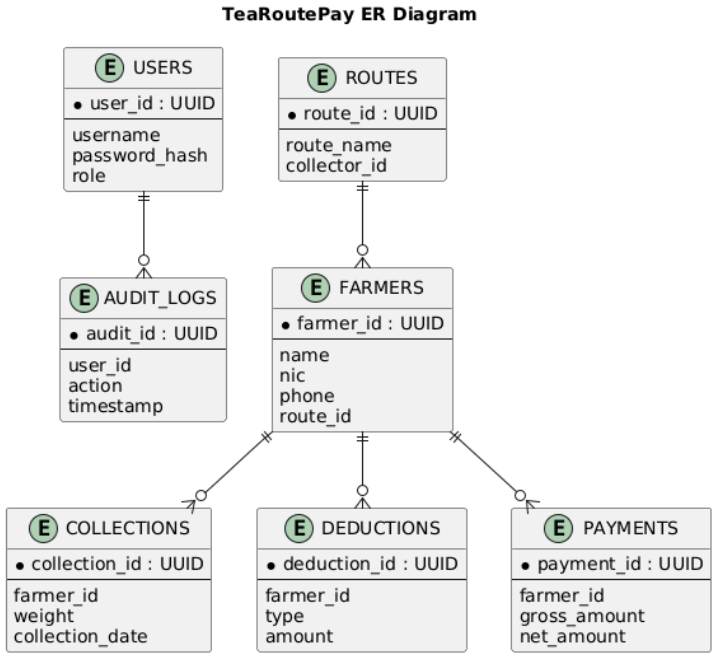

Figure 30: TeaRoutePay Central Entity-Relationship Diagram — USERS, ROUTES, FARMERS, COLLECTIONS, DEDUCTIONS, PAYMENTS, AUDIT_LOGS

The central entity-relationship model defines the authoritative relational schema held in the PostgreSQL tier. Seven entities are modelled. USERS holds principals and their roles. ROUTES holds the collection-route master data and its assigned collector. FARMERS is the central entity, carrying a foreign key to ROUTES and acting as the parent of the three transactional entities. COLLECTIONS, DEDUCTIONS, and PAYMENTS each hold a farmer_id foreign key and represent, respectively, the field weight records, the scheduled liabilities, and the computed payment outcomes. AUDIT_LOGS references USERS and captures the immutable action history.

Every primary key is a UUID, which is essential to the offline-first design: because identifiers are generated on the device at record-creation time and are globally unique, records created offline on multiple devices can be merged into the central store without key collision, which is the structural precondition for idempotent, conflict-aware synchronisation (NFR-19).

4.4.2 Mobile Local (SQLite) Schema

The collector device holds a deliberately reduced, denormalised subset of the central schema, optimised for offline read performance and synchronisation bookkeeping. It carries only what the field workflow requires, plus the synchronisation metadata that the cloud schema does not need. Three design intentions are made explicit. First, weights and monetary amounts are stored as integers (weight_grams, amount_cents), never as floats, honouring the fixed-point financial-accuracy constraint at the very point of capture (FR-37, NFR-20). Second, every transactional row carries a sync_status and a device_timestamp; the former drives the Collection Record state machine, and the latter is the key on which last-write-wins conflict resolution operates (FR-28). Third, an explicit SYNC_QUEUE with a bounded retry_count is the on-device realisation of the retry-with-back-off requirement (FR-29, NFR-21). The entire local store is encrypted at rest via SQLCipher (NFR-07).

4.4.3 Data Dictionary

The data dictionary below specifies the principal persistent attributes of the central schema, their types, constraints, and the requirements they serve. Monetary fields use a fixed-point integer representation (cents); identifiers are UUIDs to support offline-safe key generation.

Entity	Attribute	Type	Constraints	Notes / Requirements
USERS	user_id	UUID	PK, not null	Principal identifier
USERS	username	VARCHAR(64)	unique, not null	BR-27 uniqueness (FR-05)
USERS	password_hash	VARCHAR(255)	not null	Salted hash; never plaintext (NFR-07)
USERS	role	VARCHAR(20)	not null, enum	Owner/Office Staff/Collector/SysAdmin (FR-02)
USERS	status	VARCHAR(16)	not null	Active / Locked (FR-03)
ROUTES	route_id	UUID	PK, not null	Route identifier
ROUTES	route_code	VARCHAR(20)	unique, not null	(FR-14)
ROUTES	collector_id	UUID	FK → USERS, not null	Assigned collector (FR-14)
FARMERS	farmer_id	UUID	PK, not null	Encoded in QR card (FR-09)
FARMERS	nic	VARCHAR(12)	unique, not null	Unique incl. deactivated (FR-08)
FARMERS	bank_account	VARCHAR(34)	not null	Disbursement target (FR-43)

Entity	Attribute	Type	Constraints	Notes / Requirements
FARMERS	route_id	UUID	FK → ROUTES	One active route (BR-28, FR-15)
FARMERS	preferred_language	VARCHAR(8)	not null	si / ta / en (FR-49)
FARMERS	status	VARCHAR(16)	not null	Registered/Active/Suspended/Deactivated (FR-12)
COLLECTIONS	collection_id	UUID	PK, not null	Device-generated (FR-20)
COLLECTIONS	weight_grams	BIGINT	not null, ≥ 0	Fixed-point integer (FR-37, NFR-20)
COLLECTIONS	sync_status	VARCHAR(16)	not null	Created/PendingSync/Synced/Locked
COLLECTIONS	device_timestamp	TIMESTAMP	not null	Conflict key (FR-28)
DEDUCTIONS	type	VARCHAR(20)	not null	Loan/Fertilizer/Custom (FR-32–35)
DEDUCTIONS	total_amount_cents	BIGINT	not null, ≥ 0	Fixed-point (NFR-20)
DEDUCTIONS	installment_cents	BIGINT	≥ 0	Loan amortisation (FR-33)
DEDUCTIONS	balance_cents	BIGINT	≥ 0	Remaining liability (FR-33)
DEDUCTIONS	start_month	CHAR(7)	YYYY-MM	(FR-32)
PAYMENTS	gross_amount_cents	BIGINT	not null, ≥ 0	kg × rate (FR-37)
PAYMENTS	deduction_amount_cents	BIGINT	not null, ≥ 0	Σ deductions (FR-38)
PAYMENTS	net_amount_cents	BIGINT	not null	Gross − deductions (FR-38, FR-36)
PAYMENTS	status	VARCHAR(20)	not null	Locked on approval (FR-41)
PAYMENTS	transfer_status	VARCHAR(12)	nullable	Success/Pending/Failed (FR-44)
AUDIT_LOGS	audit_id	UUID	PK, not null	(NFR-10)
AUDIT_LOGS	user_id	UUID	FK → USERS, not null	Actor attribution (NFR-10)
AUDIT_LOGS	action	VARCHAR(255)	not null	Append-only (FR-06, FR-42, FR-45)
AUDIT_LOGS	timestamp	TIMESTAMP	not null	(NFR-10)

4.4.4 Architectural Data-Integrity Rules

Beyond conventional relational constraints, the design enforces four cross-cutting data-integrity rules essential to the financial trustworthiness of the system.

- **1. Maker-Checker Separation.** No single principal may both compute and authorise a disbursement. The payment calculation is performed by Office Staff (the maker, SD-13), but the approval that locks the run and releases funds is reserved to the Owner (the checker, SD-15). This separation of duties is enforced

by the role-based access control matrix (Section 7) and structurally prevents both fraud and unilateral error from reaching the bank (FR-40, FR-41, FR-02).

- **2. Synchronisation Integrity.** Reconciliation from device to centre is idempotent and conflict-aware. Because every record carries a globally unique UUID generated at creation, a record transmitted more than once is recognised and never duplicated (NFR-19); and where two devices submit competing values for the same logical record, the last-write-wins policy keyed on device_timestamp deterministically selects the winner and logs the conflict for Owner review (FR-28).
- **3. Immutable Financial History.** Once a payment run is approved, its payment records, and the collection records that fed them, transition to a Locked state and become immutable; no update or delete is permitted by any role (FR-41). The audit log is strictly append-only — entries may be written and read but never altered or removed — providing a tamper-evident record of every action affecting farmer profiles, deductions, and payments (NFR-10, FR-06, FR-42, FR-45).
- **4. Loan Amortisation Integrity.** A loan is recorded once with a total amount and a monthly installment (FR-32). During each monthly calculation, the Payment Service applies exactly one installment, decrementing the loan's balance_cents, and ceases to apply the deduction automatically once the balance reaches zero (FR-33). The invariant maintained is that the sum of installments applied across all runs never exceeds the original total, and the balance is monotonically non-increasing — preventing both over-recovery from the farmer and indefinite deduction.

4.4.5 Requirements Traceability Matrix (Functional)

The following matrix consolidates the realisation of every functional requirement by the design artefacts of this section, providing forward traceability from requirement to design.

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-01	User login	Authentication Service; User	SD-01, SD-08
FR-02	Role-based access control	Authentication Service	SD-01, SD-08, SD-19
FR-03	Account lockout	Authentication Service	SD-08

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-04	Session timeout	Auth Service / Gateway	SD-08
FR-05	User-account management	Authentication Service; User	SD-19
FR-06	Auth audit logging	Audit Service; AuditLog	SD-19
FR-07	Farmer registration	Farmer Service; Farmer	SD-09
FR-08	Unique NIC validation	Farmer Service	SD-09
FR-09	QR-code generation	Farmer Service	SD-09, SD-03
FR-10	QR-card printing	Farmer Service / Reporting	SD-09
FR-11	Profile update + audit	Farmer Service; AuditLog	SD-10
FR-12	Farmer deactivation (soft)	Farmer Service; Farmer SM	SD-10
FR-13	Farmer search / filter	Farmer Service	SD-10
FR-14	Route creation	Route Service; Route	SD-20
FR-15	Farmer-to-route assignment	Route Service	SD-20
FR-16	Route schedule	Route Service	SD-20
FR-17	Route selection at session start	Mobile App / Route cache	SD-02
FR-18	QR-code scanning	Mobile App; Collection Service	SD-03
FR-19	Manual farmer fallback	Mobile App	SD-03
FR-20	Offline weight entry + local store	Collection Service; CollectionRecord	SD-04
FR-21	Pre-sync weight correction	Collection Service	SD-04
FR-22	Duplicate-entry prevention	Collection Service	SD-04
FR-23	Session summary view	Mobile App	SD-04
FR-24	Connectivity-status indicator	Mobile App	SD-07

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-25	Dual / selectable sync mode	Sync Service	SD-07
FR-26	Local LAN sync	Sync Service	SD-07
FR-27	Cloud sync	Sync Service	SD-07
FR-28	Sync conflict resolution	Sync Service	SD-07
FR-29	Sync retry with back-off	Sync Service	SD-07
FR-30	Monthly tea-rate setting	Payment Service	SD-12
FR-31	Tea-rate history (immutable past)	Payment Service	SD-12
FR-32	Loan deduction recording	Deduction Service; Deduction	SD-11
FR-33	Automatic loan installment	Deduction / Payment Service	SD-11, SD-13
FR-34	Fertilizer deduction recording	Deduction Service	SD-05, SD-11
FR-35	Custom deduction types	Deduction Service	SD-11
FR-36	Negative-payment alert	Payment Service	SD-13
FR-37	Gross-payment calculation	Payment Service; PaymentRun	SD-13
FR-38	Net-payment calculation	Payment Service; PaymentRun	SD-13
FR-39	Payment preview	Payment Service	SD-14
FR-40	Owner payment approval	Payment Service	SD-15
FR-41	Record immutability on approval	Payment Service; State Machines	SD-15
FR-42	Approval audit record	Audit Service	SD-15
FR-43	Bank-transfer submission	Bank Integration Service	SD-16
FR-44	Transfer-status retrieval	Bank Integration Service	SD-16
FR-45	Transfer audit log	Audit Service	SD-16

FR	Requirement (abridged)	Realising Service / Class	Sequence Realisation
FR-46	Manual transfer re-submission	Bank Integration Service	SD-16
FR-47	Farmer SMS on success	SMS Service	SD-17
FR-48	SMS only after confirmation	SMS Service	SD-17
FR-49	Multi-language SMS	SMS Service	SD-17
FR-50	Daily collection report	Reporting Service	SD-18
FR-51	Monthly payment report	Reporting Service	SD-18
FR-52	Report export (PDF/CSV)	Reporting Service	SD-18
FR-53	Farmer payment receipt	Reporting Service	SD-18

5. USER INTERFACE DESIGN

5.1 UX Decisions for Low-Technical-Proficiency Users

The user-experience design is governed by the persona analysis of the SRS, which characterises the primary field user (the Collector, Suresh, 28, lorry driver) as low in technical proficiency and the primary office user (the Owner, Nimal, 52) as medium. The design optimises the field client for speed, error-resistance, and minimal typing, and optimises the office client for familiar, tabular, dashboard-driven control. The following decisions implement the usability requirements NFR-12 through NFR-16.

- **Large touch targets.** Every interactive element on the mobile client meets a minimum target of 48x48 dp, sized for one-handed use by a standing collector, often gloved and in bright outdoor light (NFR-12). Primary actions — Scan, Save, Sync — are rendered as oversized, high-contrast buttons.
- **Minimal-typing flows.** The core collection task is reduced to a three-step flow — scan, weigh, save — in which QR scanning replaces manual farmer identification entirely (FR-18) and numeric weight entry is the only free input. Field deductions and advances use quick-select menus rather than free text. This directly serves the 30-minute training target of NFR-13.
- **Offline indicators.** A persistent connectivity indicator displays the current state — Offline, LAN Connected, or Internet Connected — and a visible pending-sync counter shows how many records remain unsynced (FR-24). This makes the system's offline behaviour legible to the user, who must trust that an unsynced record is nonetheless safe.
- **Plain-language, corrective error messages.** All error messages are written in plain language with explicit corrective guidance rather than codes or technical jargon (NFR-15).
- **Localisation.** Both clients support Sinhala, Tamil, and English, with the language selectable per user; SMS content is additionally localised to each farmer's preferred language (NFR-14, FR-49).

- **Familiar office conventions.** The desktop/web client follows standard conventions — keyboard navigation, tab order, tabular data entry, and a single-click approval flow — so that no skill beyond ordinary desktop usage is required (NFR-16).

The principal mobile screens are: Login, Route Selection, QR Scan, Weight Entry, Offline Sync status, Advance Payment, and Session Summary. The principal office screens are: Dashboard, Farmer Management, Route Management, Deductions, Tea-Rate Setup, Payment Calculation, Approval Screen, Reports, and User Management.

5.1.1 Mobile Application Wireframes

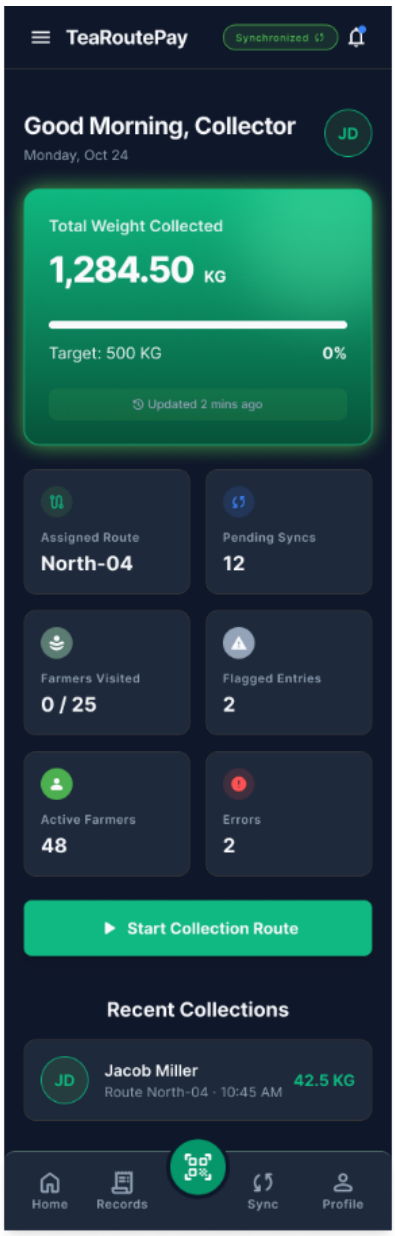

Figure 1: Mobile Collector Dashboard

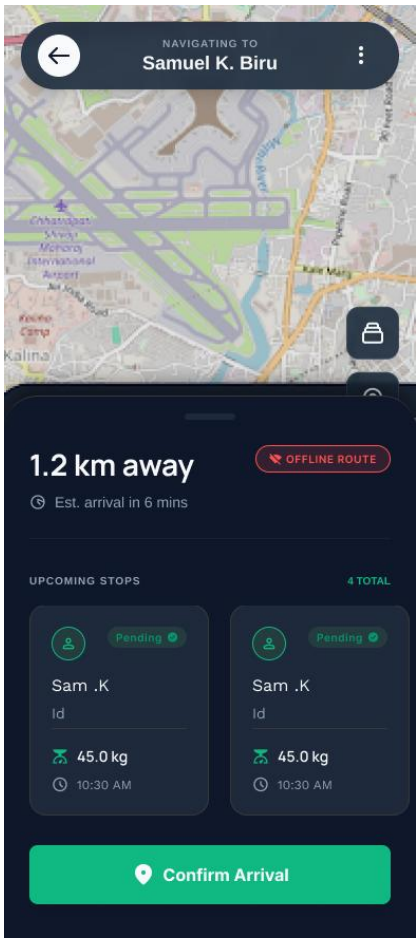

Figure 2: Route Map

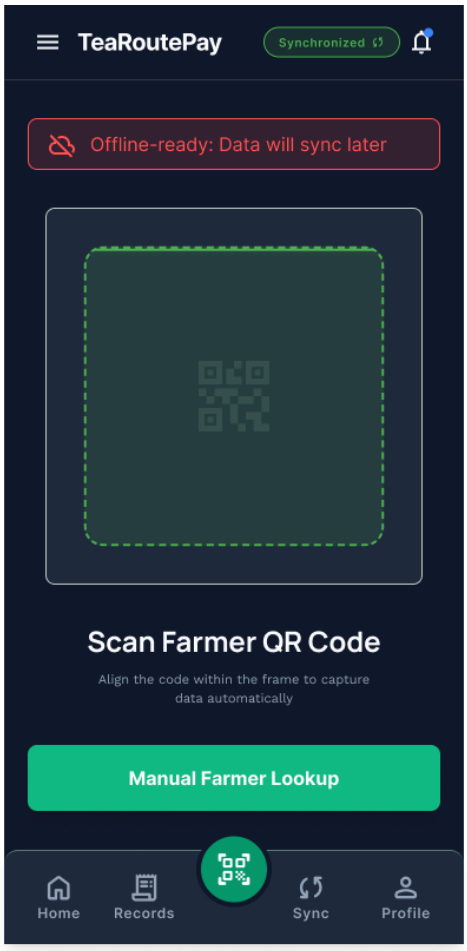

Figure 3: QR Scanner

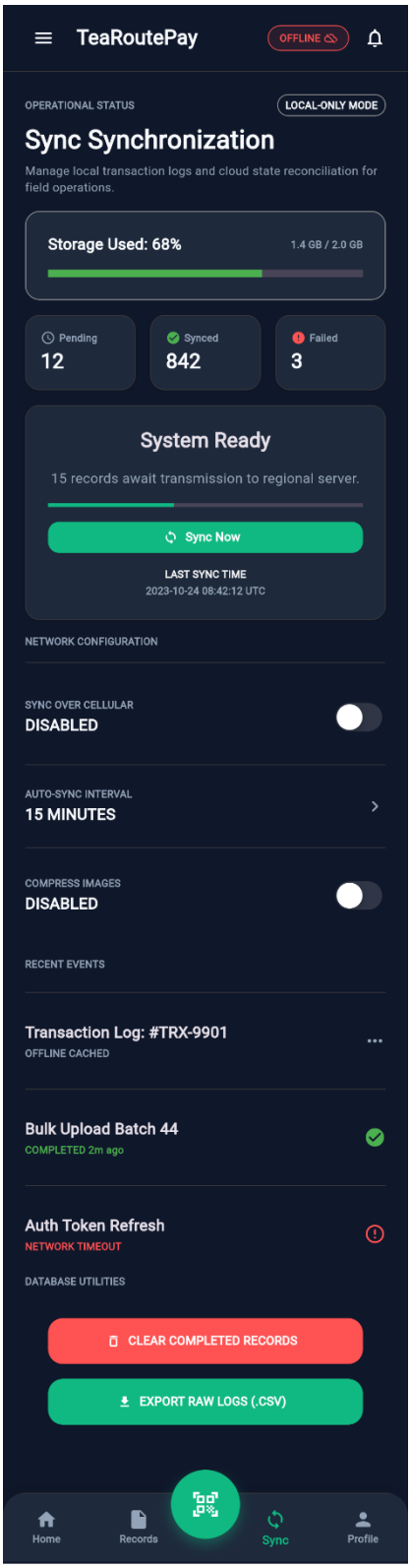

Figure 4: Sync Config Screen

Figure 5: Farmer Manual Search

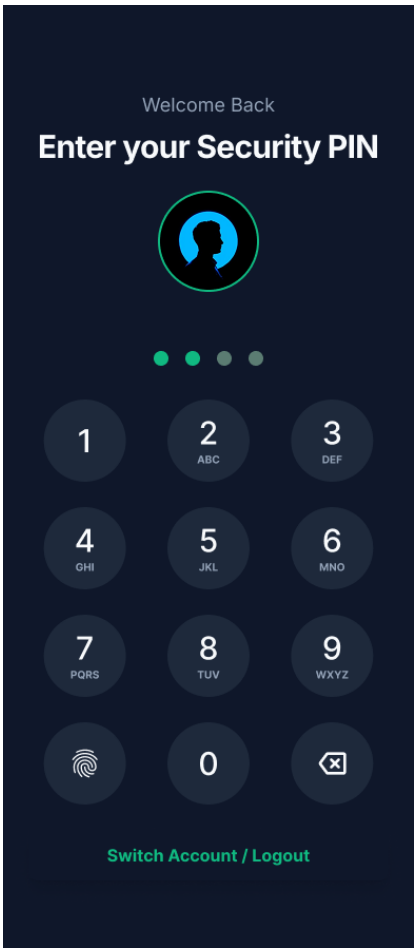

Figure 6: Authentication Pin

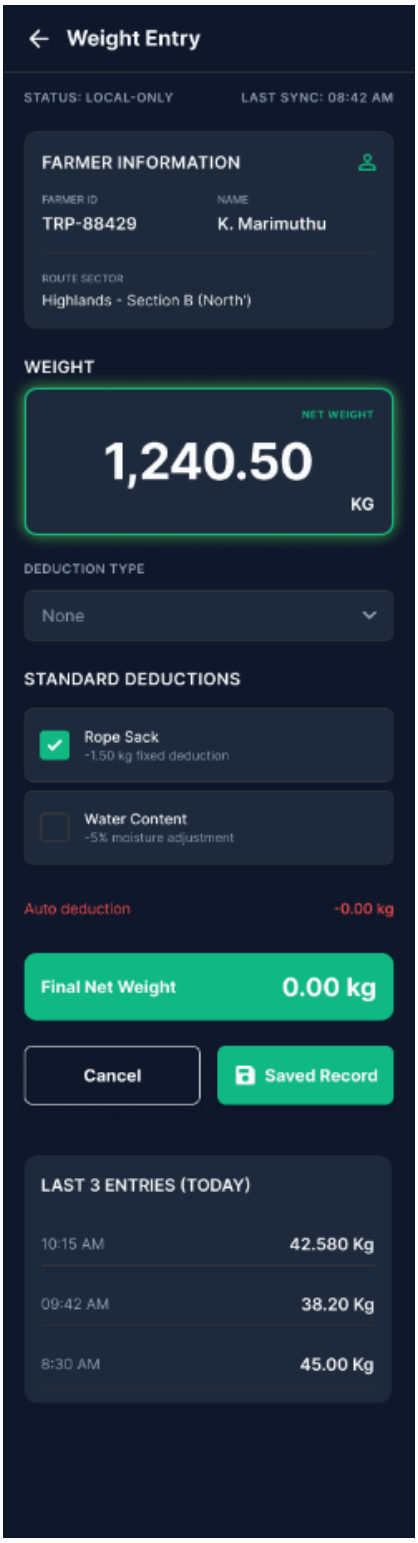

Figure 7: Weight Entry

5.1.2 Office Desktop Application Wireframes

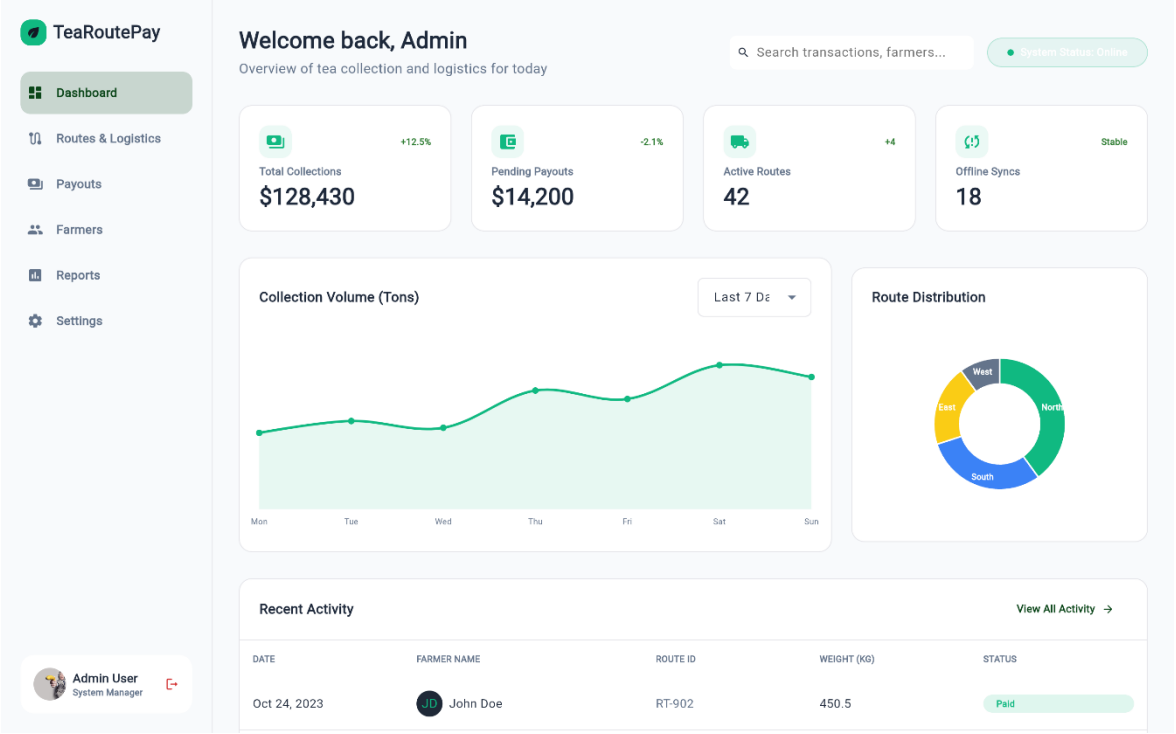

Figure 1: Admin Dashboard

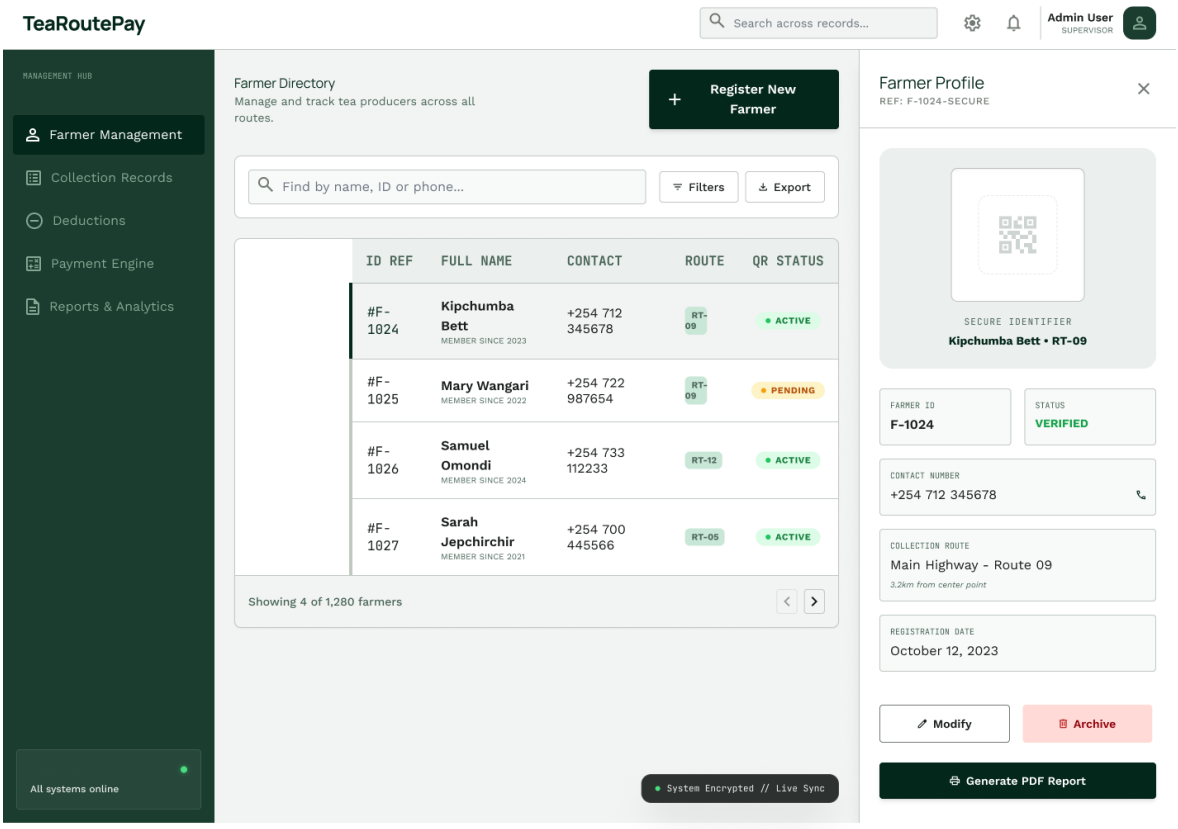

Figure 2: Farmer Details

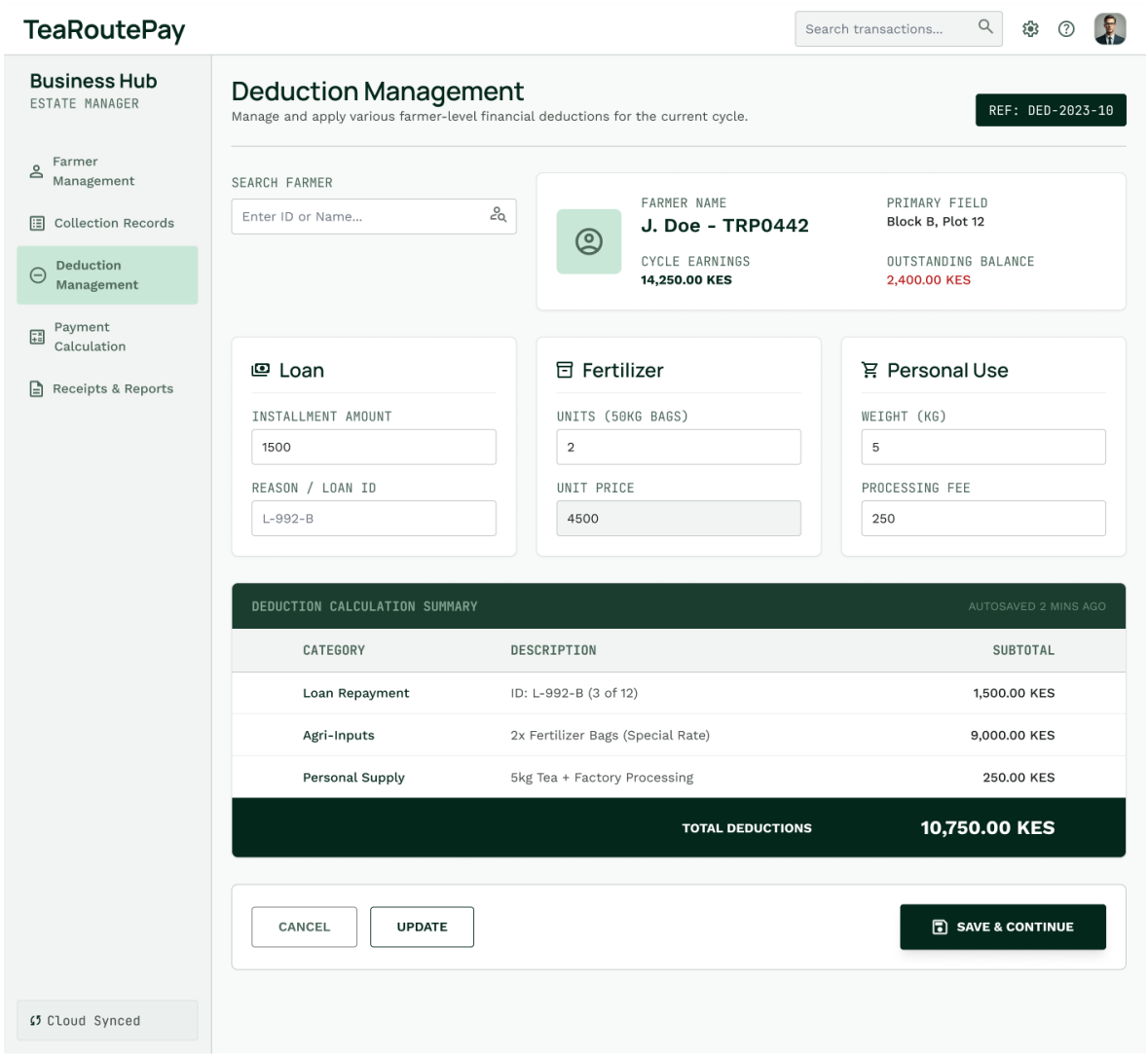

Figure 3: Deduction Management

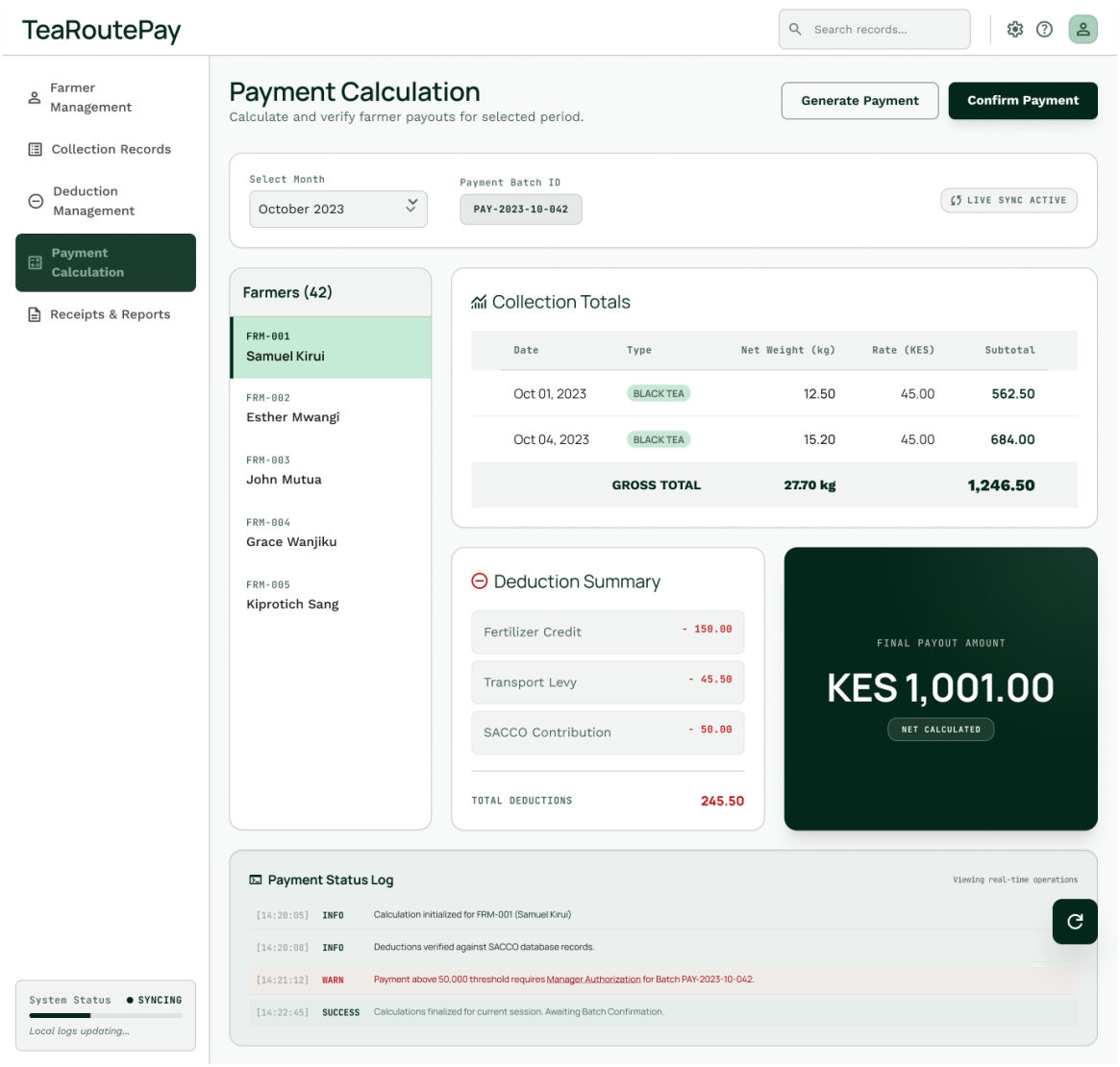

Figure 4: Payment Calculation

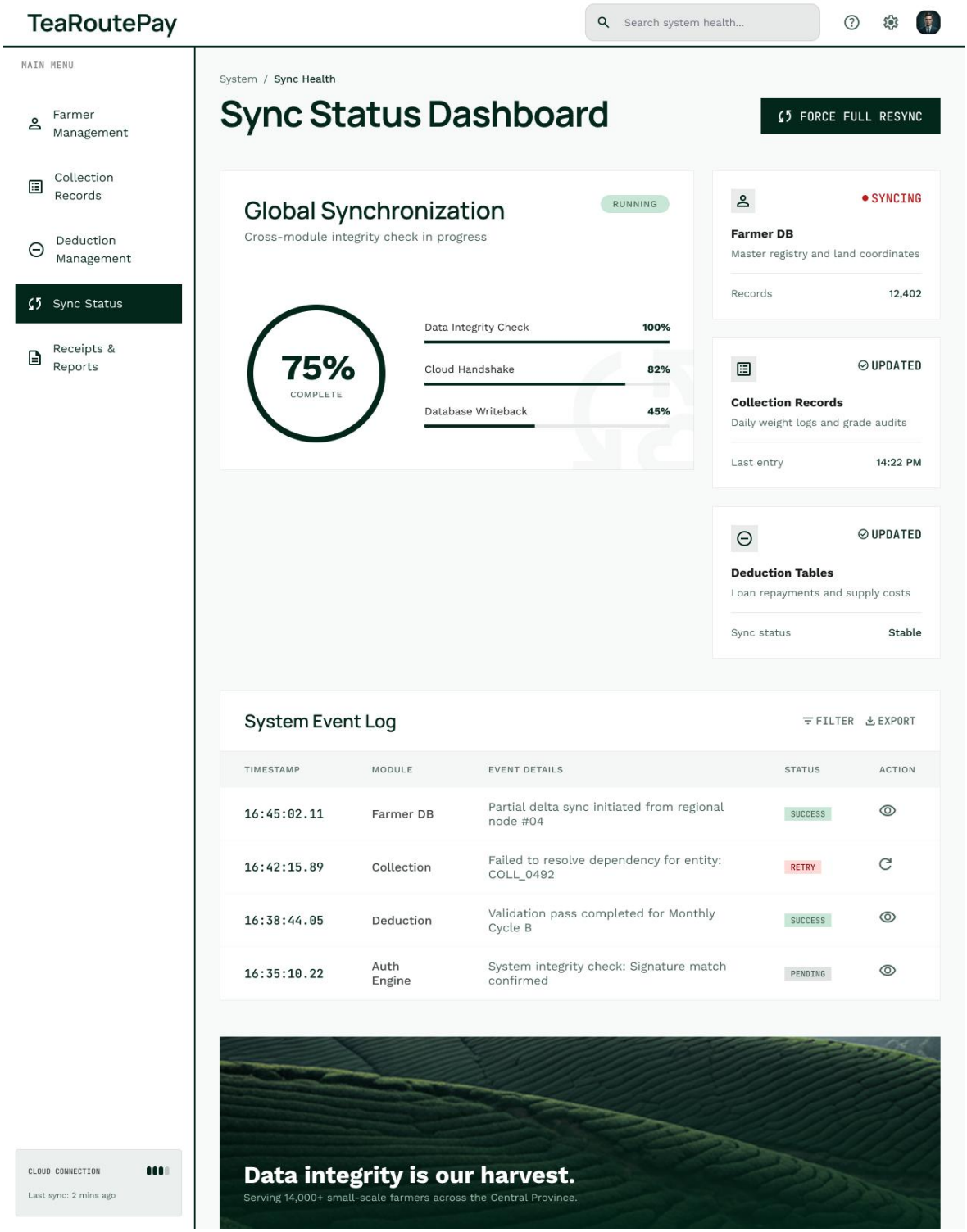

Figure 5: Sync Status Dashboard

5.2 Navigation Flow

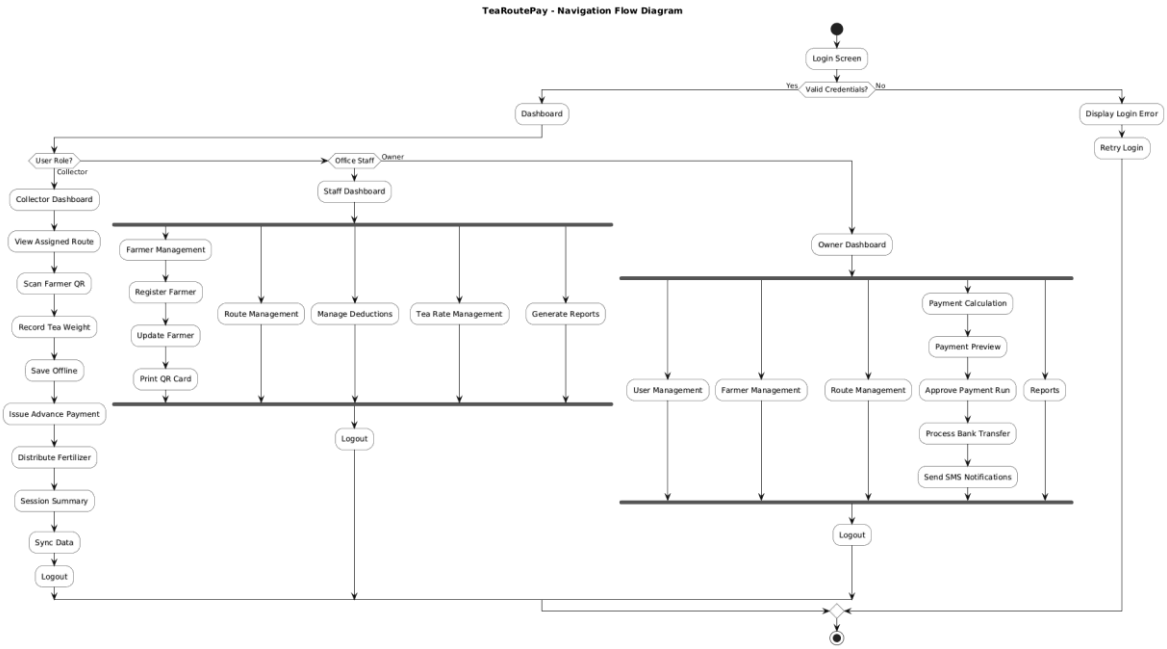

Figure 31: TeaRoutePay Navigation Flow Diagram — role-branched journeys for Collector, Office Staff, and Owner

The navigation model branches at authentication on the authenticated principal's role, producing three distinct journeys from a single login surface. After the Login Screen, a valid-credentials check routes a failure to Display Login Error and Retry Login, and a success to the Dashboard, where the role discriminator selects the journey.

The Collector journey is a linear, top-to-bottom field workflow optimised for repetition: Collector Dashboard → View Assigned Route → Scan Farmer QR → Record Tea Weight → Save Offline → Issue Advance Payment → Distribute Fertilizer → Session Summary → Sync Data → Logout. The linearity is deliberate — it mirrors the physical sequence of a route stop and minimises navigational decisions for the low-proficiency user. The Save Offline step sits in the critical path precisely because persistence must precede, and is independent of, the later Sync Data step.

The Office Staff journey fans out from a Staff Dashboard to the parallel administrative capabilities — Farmer Management (with its Register Farmer, Update Farmer, and Print QR Card sub-flow), Route Management, Manage Deductions, Tea Rate Management, and Generate Reports — before converging on Logout. The parallel structure reflects that these are independent tasks selected on demand rather than a fixed sequence.

The Owner journey extends the staff capabilities with the privileged financial path: from the Owner Dashboard, the Owner reaches Payment Calculation → Payment Preview → Approve Payment Run → Process Bank Transfer → Send SMS Notifications, alongside User Management, Farmer Management, Route Management, and Reports. The placement of approval and disbursement exclusively within the Owner journey is the navigational expression of the maker-checker control: the approval screen is simply unreachable for a non-Owner principal.

5.3 UI Resources and Prototyping

The complete UI design system, including interactive components, assets, and low-fidelity prototypes, is maintained in the project's design workspace. The panel and supervisors may access the live Figma file via the link below:

- **TeaRoutePay Figma Design Workspace:** https://www.figma.com/design/t7JCaM3YTcm3Az4QTlPWwW/TeaRoutePay

Note: The live workspace contains the most current version of the mobile and desktop wireframes; individual screen exports referenced as figures in this section were captured from this workspace at the time of document publication.

6. INTERFACE DESIGN

6.1 External Interfaces

6.1.1 Bank Transfer API

The Bank Integration Service disburses approved payments through the external Bank Transfer API. Communication is over HTTPS (TLS 1.3), authenticated with a bearer credential retrieved from the secrets manager and never embedded in source (NFR-09). A representative disbursement request and response are detailed below.

Element	Value	Notes
Method	POST	
Path	/api/transfers	(FR-43)
Header: Authorization	Bearer <token>	Retrieved from secrets manager (NFR-09)
Header: Content-Type	application/json	
Header: Idempotency-Key	UUID	Prevents duplicate disbursement on retry (NFR-19, NFR-21)
Body: paymentRunId	UUID	Identifies the approved run
Body: farmerId	UUID	Payee identifier
Body: accountNumber	VARCHAR	Farmer bank account (FR-43)
Body: amountCents	BIGINT	Integer cents — no floating point (FR-37, NFR-20)
Body: currency	"LKR"	Sri Lankan Rupee
Body: reference	"TRP-YYYY-MM-UUID"	Traceable payment reference
Response: transferId	"BNK-NNNNN"	Bank's transfer identifier (FR-44)
Response: status	SUCCESS / PENDING / FAILED	Mapped to internal taxonomy (FR-44)
Response: processedAt	ISO 8601 timestamp	Written to audit log (FR-45)

The Idempotency-Key is essential: should the request be retried after a network timeout, the bank must treat the repeat as the same logical transfer and not disburse twice — the external counterpart to the system's internal idempotency guarantee (NFR-19, NFR-21). Amounts are transmitted as integer cents, never as decimals, consistent with the fixed-point financial-accuracy constraint. A Failed or timed-out transfer is retried with exponential back-off up to three attempts, after which it is surfaced for manual re-submission (FR-46).

6.1.2 SMS Gateway API

After a transfer is confirmed successful — and only then (FR-48) — the SMS Service dispatches a localised confirmation through the SMS Gateway API.

Element	Value	Notes
Method	POST	
Path	/api/send	(FR-47)
Header: Authorization	Bearer <token>	Secrets manager (NFR-09)
Body: to	"+9477XXXXXXX"	Farmer's registered phone (FR-07)
Body: language	"si" / "ta" / "en"	Farmer's preferred language (FR-49)
Body: templateId	"PAYMENT_CONFIRM"	Localised template
Body: params.amountLKR	String (formatted)	Net payment amount
Body: params.month	"YYYY-MM"	Payment month
Response: messageId	"SMS-NNNNN"	Delivery tracking reference
Response: status	QUEUED / SENT / FAILED	Tracked for reporting

6.2 Internal Interfaces

Internal communication between the clients and the backend traverses the Spring Cloud API Gateway over HTTPS, carrying the bearer JWT issued at login. The principal internal endpoints, grouped by owning service, are as follows.

Service	Method & Path	Purpose	Requirement
Farmer	POST /farmers	Register farmer	FR-07
Farmer	GET /farmers/{id}	Retrieve profile	FR-13
Farmer	PUT /farmers/{id}	Update profile	FR-11
Collection	POST /collections	Ingest a collection record	FR-20
Collection	GET /collections/sync	Pull/confirm sync set	FR-26, FR-27
Payment	POST /payments/calculate	Run monthly calculation	FR-37, FR-38
Payment	POST /payments/approve	Approve and lock a run	FR-40, FR-41
Deduction	POST /deductions	Record loan/fertilizer/custom deduction	FR-32, FR-34
Route	POST /routes	Create collection route	FR-14
Auth	POST /auth/login	Issue JWT	FR-01

Service	Method & Path	Purpose	Requirement
Reporting	GET /reports/daily/{date}	Daily collection report	FR-50
Reporting	GET /reports/payment/{month}	Monthly payment report	FR-51

6.2.1 Synchronisation Payload

The mobile-to-backend synchronisation payload is the most design-significant internal interface, because it must carry the data and the metadata required for idempotent, conflict-aware reconciliation. The collector device submits a batch of pending records to POST /collections/sync, each bearing its device-generated UUID and device timestamp.

Payload Field	Type	Description	Requirement
deviceId	String	Identifies the submitting device	FR-28
syncMode	Enum	LOCAL_FIRST_CLOUD_BACKUP / LOCAL / CLOUD	FR-25
records[].collectionId	UUID	Device-generated record identity	NFR-19
records[].farmerId	UUID	Farmer being recorded	FR-18
records[].weightGrams	Integer	Fixed-point weight (no floats)	FR-37, NFR-20
records[].collectionDate	ISO date	Calendar date of collection	FR-20
records[].deviceTimestamp	ISO 8601	Device clock timestamp for conflict resolution	FR-28
response.accepted	UUID[]	Records accepted; client advances to Synced	NFR-19
response.duplicatesIgnored	UUID[]	Records already present; idempotency confirmed	NFR-19
response.conflicts[].resolution	String	LAST_WRITE_WINS for contested records	FR-28

The response is the explicit realisation of synchronisation integrity: duplicatesIgnored confirms idempotency (a UUID already present is acknowledged, not re-inserted, NFR-19), and conflicts reports each contested record together with the deterministic resolution applied and logged for Owner review (FR-28). The client uses the accepted list to advance the corresponding local records from PendingSync to Synced.

7. SECURITY DESIGN

7.1 Authentication and Authorisation

Authentication is realised through the OAuth2 bearer-token model with JSON Web Tokens. On successful credential validation, the Authentication Service issues a JWT signed with RS256, an asymmetric algorithm that allows every downstream service to verify a token using the public key without ever holding the private signing key (FR-01, NFR-08). Tokens expire within the configured session timeout (FR-04), and the gateway rejects expired or malformed tokens at the perimeter. Account lockout after five failed attempts (FR-03) and audit logging of every authentication event (FR-06) complete the authentication design.

Authorisation is enforced through Role-Based Access Control over the four roles defined by FR-02, each with a distinct, non-overlapping permission set. The matrix below specifies the authoritative capability assignment; it is the structural enforcement of the maker-checker rule — note that Approve Payment Run is granted to the Owner alone.

Capability	Owner	Office Staff	Collector	Sys Admin
Field collection (scan, weigh, save)	—	—	Yes	—
Field deduction / advance	—	—	Yes	—
Synchronise collection data	—	Yes	Yes	—
Register / update farmer	Yes	Yes	Yes (field)	—
Deactivate farmer	Yes	Yes	—	—
Manage routes	Yes	Yes	—	—
Record deductions	Yes	Yes	—	—
Set monthly tea rate	Yes	Yes	—	—
Run payment calculation (maker)	Yes	Yes	—	—
Approve payment run (checker)	**Yes**	**—**	**—**	**—**
Process / re-submit bank transfer	Yes	Yes	—	—
Generate reports	Yes	Yes	—	—
Manage user accounts and roles	Yes	—	—	Yes
Configure system parameters	Yes	—	—	Yes

7.2 Data Protection

Data is protected both at rest and in transit, addressing NFR-06 and NFR-07.

- **At rest (server).** Farmer personal and financial data in the central PostgreSQL tier is encrypted using AES-256. Database credentials and all API keys are held in a secrets manager (AWS Secrets Manager or HashiCorp Vault) and are never committed to source control (NFR-09).
- **At rest (device).** The mobile SQLite store is encrypted with SQLCipher, which transparently applies AES-256 to the entire local database file. This is the critical control for the offline tier: a lost or stolen device does not expose collection or farmer data, because the at-rest data on the device is encrypted to the same standard as the server (NFR-07).
- **In transit.** All network communication — client-to-gateway, inter-service, and outbound to the Bank and SMS gateways — uses TLS 1.3 (the SRS mandates TLS 1.2 or higher; TLS 1.3 is selected as the stronger conforming choice) (NFR-06).
- **Input validation.** All inputs are validated server-side against DTO constraints; persistence uses parameterised queries exclusively, eliminating SQL-injection vectors, and output encoding prevents cross-site scripting on the web client.

7.3 Auditability

The audit design provides the immutable, attributable record required for financial compliance and dispute resolution (NFR-10). The Audit Service consumes events from the RabbitMQ bus and writes them to an append-only store: entries may be created and read but never updated or deleted by any role, through any interface. Every entry records the acting user, the action, the affected entity, the before/after values where applicable, and a timestamp — making every change to a farmer profile, a deduction, or a payment record traceable by user, date, and time. Because approved payment and collection records are themselves immutable (Section 4.4.4) and the audit log is append-only, the financial history is tamper-evident end to end: a discrepancy can always be attributed, and no actor can silently rewrite the past (FR-06, FR-42, FR-45).

7.4 OWASP Top 10 Mitigations

The design's defences are mapped to the OWASP Top 10 (2021) threat categories below.

OWASP Threat	TeaRoutePay Mitigation	Supporting Requirement
A01 Broken Access Control	RBAC matrix enforced at gateway and service level; maker-checker separation on payment approval	FR-02, FR-40, FR-41
A02 Cryptographic Failures	AES-256 at rest (PostgreSQL server and SQLCipher on device); TLS 1.3 in transit; RS256 JWT signing	NFR-06, NFR-07, NFR-08
A03 Injection	Parameterised queries throughout; server-side DTO validation; output encoding on web client	NFR-18
A04 Insecure Design	Threat-modelled architecture; immutable financial records; offline-first failure-safe handling	FR-41, NFR-17
A05 Security Misconfiguration	Hardened, minimal container images; externalised and reviewed configuration; no secrets in code	NFR-25, NFR-27
A06 Vulnerable and Outdated Components	Automated dependency scanning in CI/CD pipeline; library version pins reviewed at each build	NFR-25
A07 Identification and Authentication Failures	JWT with short expiry; RS256 signing; account lockout after 5 failures; configurable session timeout	FR-03, FR-04, NFR-08
A08 Software and Data Integrity Failures	Append-only immutable audit log; cryptographic locking of all approved payment records	NFR-10, NFR-11
A09 Security Logging and Monitoring Failures	Centralised Audit Service; structured JSON logs across all services; alerting on payment-service downtime	NFR-10, NFR-29
A10 Server-Side Request Forgery	Egress network filtering; strict allow-list of external endpoints (Bank API, SMS Gateway only)	NFR-06

8. NON-FUNCTIONAL DESIGN DECISIONS

This section consolidates the mapping of every non-functional requirement of the SRS (NFR-01 to NFR-29) to the specific architectural or detailed-design mechanism that satisfies it, providing the backward traceability that completes the requirements baseline.

NFR	Requirement (abridged)	Design Solution	Where Realised
NFR-01	QR scan and save weight < 1 s	Read/write served entirely from on-device SQLite; no network in the field path	§2.1, §3.5, SD-04
NFR-02	Payment report (500 farmers) < 5 s	Indexed queries; pre-computed payment results read by Reporting Service	§3.6, SD-14, SD-18
NFR-03	Full-day sync (500 records) < 30 s	Batched synchronisation payload over LAN; idempotent bulk ingest	§3.1, SD-07, §6.2.1
NFR-04	Payment calc (1,000 farmers) < 10 s	Aggregation queries; deterministic fixed-point engine in Payment Service	SD-13, §4.4.4
NFR-05	≥ 10 concurrent desktop sessions	Stateless services behind load-balanced gateway; Kubernetes horizontal scaling	§3.5
NFR-06	TLS 1.2+ for all communications	TLS 1.3 on all client, inter-service, and external links	§7.2
NFR-07	AES-256 encryption at rest	AES-256 on server; SQLCipher (AES-256) on device local store	§7.2
NFR-08	JWT RS256, expiring	RS256-signed JWT issued by Auth Service; expiry enforced at gateway	§7.1, SD-01
NFR-09	Secrets in secrets manager	Secrets manager for DB, Bank, and SMS credentials; none in source control	§6.1, §7.2
NFR-10	Immutable audit log	Append-only Audit Service fed by RabbitMQ event bus	§3.2, §7.3
NFR-11	Locked approved records	State-machine lock on approval; cryptographic record locking	§4.3.2, §7.4
NFR-12	Touch targets ≥ 48x48 dp	Oversized high-contrast controls on mobile client	§5.1
NFR-13	≤ 30 min collector training	Three-step scan→weigh→save flow; quick-select menus for deductions	§5.1, SD-04
NFR-14	Sinhala/Tamil/English UI	Per-user localisation on both clients and per-farmer language for SMS	§5.1

NFR	Requirement (abridged)	Design Solution	Where Realised
NFR-15	Plain-language error messages	Corrective, jargon-free error messaging on all screens	§5.1
NFR-16	Standard desktop conventions	Keyboard nav, tab order, tabular entry, standard shortcut keys on web client	§5.1
NFR-17	Zero field data loss	Unconditional local write before sync; durable transactional SQLite store	§2.1, §4.3.3, SD-04
NFR-18	ACID transactions	PostgreSQL with Spring @Transactional boundaries on all write paths	§3.6
NFR-19	Idempotent sync	UUID primary keys generated on device; duplicate detection on central ingest	§4.4.4, §6.2.1
NFR-20	Deterministic calculation	Fixed-point integer arithmetic; pure deterministic engine in Payment Service	§4.1, §4.4.3, SD-13
NFR-21	Retry with back-off (max 3)	Bounded retry queue on device sync; exponential back-off on external API calls	§4.4.2, SD-16
NFR-22	≥ 2,000 farmer records without degradation	Indexed PostgreSQL; database-per-service partitioning; horizontal scaling	§3.2, §3.5
NFR-23	≥ 5 concurrent collector syncs	Stateless, containerised ingest endpoints behind load balancer	§3.1, SD-07
NFR-24	Horizontal scaling without code change	Stateless Spring Boot services on Kubernetes with declarative HPA	§3.5
NFR-25	Documented coding standards and CI/CD	Containerised builds; GitHub Actions CI/CD; peer-reviewed PRs	§3.6
NFR-26	Schema migrations with rollback	Versioned migration framework (Flyway) per service schema	§3.2
NFR-27	Configurable params via admin UI	Externalised configuration for rates, timeouts, and SMS templates	§3.1, §7.4
NFR-28	≥ 90% unit-test coverage (payment logic)	Pure, deterministic calculation engine designed for isolated unit testing	§4.1, SD-13
NFR-29	Structured JSON logs	JSON-structured application logs across all services; compatible with Grafana/ELK	§7.4

APPENDIX A — ARCHITECTURAL DECISION RECORDS

Architectural Decision Records (ADRs) document the significant design decisions, the context that made them necessary, the alternatives considered, and the consequences accepted. They are maintained in the repository under documents/architecture/decisions/ and are reproduced here for completeness.

ADR-001: Adopt a Microservices Architecture

Field	Content
Status	Accepted
Context	The backend must scale the payment workload independently, isolate faults, and integrate an offline synchronisation subsystem with different availability characteristics from the transactional core.
Decision	Decompose the backend into independently deployable Spring Boot services, one per bounded capability, fronted by a Spring Cloud API Gateway.
Rationale	Supports independent scaling, fault isolation, and maintainability. Each service can be deployed, tested, and modified without disturbing the others. Addresses NFR-22 to NFR-26.
Alternatives	Monolithic (rejected — tight coupling impedes independent scaling and offline-sync integration); Layered/n-tier (rejected — does not accommodate the distributed field/office/cloud topology with differing availability profiles).
Consequences	Greater operational complexity (orchestration, observability, network latency between services) is accepted in exchange for independent scalability, fault isolation, and maintainability.

ADR-002: Use PostgreSQL as the Central Database

Field	Content
Status	Accepted
Context	The system processes financial transactions requiring strict correctness; payment records, once approved, must be cryptographically reliable and ACID-safe.
Decision	Use PostgreSQL with a schema-per-service ownership model. Each microservice owns its schema; no service may query another's schema directly.
Rationale	Full ACID compliance is the correctness foundation for monetary processing (NFR-18). Per-service schemas preserve architectural isolation while keeping one database platform to operate, back up, and migrate.
Alternatives	MySQL/MariaDB (rejected — weaker MVCC semantics); MongoDB (rejected — document model does not enforce the relational integrity required for payment runs and deductions).
Consequences	Strong transactional guarantees; standardised migration tooling (Flyway); single database platform for infrastructure operations. Join queries spanning service boundaries require API calls rather than SQL JOINs.

ADR-003: Use RabbitMQ for Asynchronous Event-Driven Communication

Field	Content
Status	Accepted
Context	External integrations (Bank API, SMS Gateway) and the Audit Service must not block or destabilise the core payment transaction; they must survive transient failures independently.
Decision	Publish financial events to RabbitMQ topics; consume them in the Bank Integration, SMS, and Audit services.
Rationale	Decoupling absorbs external latency and transient failure through consumer-side retry and back-off. The Audit Service consumes the same event stream, providing an immutable, ordered record of all financial events (NFR-10, NFR-21).
Alternatives	Synchronous HTTP calls (rejected — a slow bank API would block payment completion); Apache Kafka (considered — offers stronger ordering guarantees but higher operational complexity; RabbitMQ is sufficient for this scale).
Consequences	Eventual consistency for side-effects (bank transfer, SMS) is accepted. Reliability and auditability of those side-effects are strengthened by the durable queue.

ADR-004: Offline-First Mobile Architecture with Encrypted SQLite

Field	Content
Status	Accepted
Context	Field collection occurs where connectivity is intermittent or absent, yet no field datum may be lost; synchronisation must be reliable and conflict-aware when connectivity resumes.
Decision	Treat the on-device SQLite store (encrypted via SQLCipher) as the primary write path. Treat synchronisation as a deferred, idempotent, conflict-aware, bounded-retry concern.
Rationale	Realises FR-20, NFR-17 (zero loss), NFR-01 (sub-second interaction), and NFR-07 (encryption at rest on device). UUID-keyed records enable offline-safe identity generation and idempotent reconciliation (NFR-19).
Alternatives	Online-only mobile (rejected — fundamentally incompatible with rural operating environment); service worker / local-first web (rejected — camera access and native performance required for QR scanning).
Consequences	Synchronisation complexity (UUID keys, last-write-wins conflict resolution, bounded retry queue, sync-status lifecycle on every record) is accepted as the necessary cost of field operational autonomy.

— End of Document —